



中山大學
SUN YAT-SEN UNIVERSITY

《计算机组成原理实验》 实验报告

(项目一)

学 院 名 称 : 计算机学院

专业 (班级) : 计算机科学与技术 9 班

学 生 姓 名 : 张玉瑶

学 号 : 23336316

时 间 : 2024 年 12 月 9 日

项目一：单周期CPU设计与实现

(套话, 可折叠)

一 实验目的

- (1) 掌握单周期 CPU 数据通路图的构成、原理及其设计方法;
- (2) 掌握单周期 CPU 的实现方法, 代码实现方法;
- (3) 认识和掌握指令与 CPU 的关系;
- (4) 掌握测试单周期 CPU 的方法;
- (5) 掌握把 CPU 烧到 Basys3 上的方法。

二 实验内容

设计一个单周期 CPU, 该 CPU 至少能实现以下指令功能操作。指令与格式如下:

==> 算术运算指令

(1) add rd, rs, rt

000000	rs(5 位)	rt(5 位)	rd(5 位)	00000 100000
--------	---------	---------	---------	--------------

功能: $GPR[rd] \leftarrow GPR[rs] + GPR[rt]$ 。

(2) sub rd, rs, rt

000000	rs(5 位)	rt(5 位)	rd(5 位)	00000 100010
--------	---------	---------	---------	--------------

功能: $GPR[rd] \leftarrow GPR[rs] - GPR[rt]$ 。

(3) addiu rt, rs, immediate

001001	rs(5 位)	rt(5 位)	immediate(16 位)	
--------	---------	---------	-----------------	--

功能: $GPR[rt] \leftarrow GPR[rs] + \text{sign_extend}(\text{immediate})$; immediate 做符号扩展再参加“+”运算。

==> 逻辑运算指令

(4) andi rt, rs, immediate

001100	rs(5 位)	rt(5 位)	immediate(16 位)	
--------	---------	---------	-----------------	--

功能: $GPR[rt] \leftarrow GPR[rs] \text{ and } \text{zero_extend}(\text{immediate})$; immediate 做 0 扩展再参加“与”运算。

(5) and rd, rs, rt

000000	rs(5 位)	rt(5 位)	rd(5 位)	00000 100100
--------	---------	---------	---------	--------------

功能: $GPR[rd] \leftarrow GPR[rs] \text{ and } GPR[rt]$ 。

(6) ori rt, rs, immediate

001101	rs(5 位)	rt(5 位)	immediate(16 位)	
--------	---------	---------	-----------------	--

功能: $GPR[rt] \leftarrow GPR[rs] \text{ or } \text{zero_extend}(\text{immediate})$ 。

(7) or rd, rs, rt

000000	rs(5 位)	rt(5 位)	rd(5 位)	00000 100101
--------	---------	---------	---------	--------------

功能: $GPR[rd] \leftarrow GPR[rs] \text{ or } GPR[rt]$ 。

==>移位指令

(8) sll rd, rt, sa

000000	00000	rt (5 位)	rd (5 位)	sa (5 位)	000000
--------	-------	----------	----------	----------	--------

功能: $\text{GPR}[\text{rd}] \leftarrow \text{GPR}[\text{rt}] \ll \text{sa}$ 。

==>比较指令

(9) slti rt, rs, **immediate** 带符号数

001010	rs (5 位)	rt (5 位)	immediate (16 位)
--------	----------	----------	-------------------------

功能: if $\text{GPR}[\text{rs}] < \text{sign_extend}(\text{immediate})$ $\text{GPR}[\text{rt}] = 1$ else $\text{GPR}[\text{rt}] = 0$ 。

==> 存储器读/写指令

(10) sw rt, **offset** (rs) 写存储器

101011	rs (5 位)	rt (5 位)	offset (16 位)
--------	----------	----------	----------------------

功能: $\text{memory}[\text{GPR}[\text{base}] + \text{sign_extend}(\text{offset})] \leftarrow \text{GPR}[\text{rt}]$ 。

(11) lw rt, **offset** (rs) 读存储器

100011	rs (5 位)	rt (5 位)	offset (16 位)
--------	----------	----------	----------------------

功能: $\text{GPR}[\text{rt}] \leftarrow \text{memory}[\text{GPR}[\text{base}] + \text{sign_extend}(\text{offset})]$ 。

==> 分支指令

(12) beq rs, rt, **offset**

000100	rs (5 位)	rt (5 位)	offset (16 位)
--------	----------	----------	----------------------

功能: if $(\text{GPR}[\text{rs}] = \text{GPR}[\text{rt}])$ $\text{pc} \leftarrow \text{pc} + 4 + \text{sign_extend}(\text{offset}) \ll 2$
else $\text{pc} \leftarrow \text{pc} + 4$

特别说明: offset 是从 PC+4 地址开始和转移到的指令之间指令条数。offset 符号扩展之后左移 2 位再相加。为什么要左移 2 位? 由于跳转到的指令地址肯定是 4 的倍数 (每条指令占 4 个字节), 最低两位是 “00”, 因此将 **offset** 放进指令码中的时候, 是右移了 2 位的, 也就是以上说的 “指令之间指令条数”。

(13) bne rs, rt, **offset**

000101	rs (5 位)	rt (5 位)	offset (16 位)
--------	----------	----------	----------------------

功能: if $(\text{GPR}[\text{rs}] \neq \text{GPR}[\text{rt}])$ $\text{pc} \leftarrow \text{pc} + 4 + \text{sign_extend}(\text{offset}) \ll 2$
else $\text{pc} \leftarrow \text{pc} + 4$

(14) blez rs, **offset**

000110	rs (5 位)	00000	offset (16 位)
--------	----------	-------	----------------------

功能: if $(\text{GPR}[\text{rs}] \leq 0)$ $\text{pc} \leftarrow \text{pc} + 4 + \text{sign_extend}(\text{offset}) \ll 2$
else $\text{pc} \leftarrow \text{pc} + 4$ 。

==>跳转指令

(15) j addr

000010	addr (26 位)				
--------	--------------------	--	--	--	--

功能: $\text{PC} \leftarrow \{\text{PC}[31:28], \text{addr}, 2' \text{b} 0\}$, 无条件跳转。

说明: 由于 MIPS32 的指令代码长度占 4 个字节, 所以指令地址二进制数最低 2 位均为 0, 将指令地址放进指令代码中时, 可省掉! 这样, 除了最高 6 位操

作码外，还有 26 位可用于存放地址，事实上，可存放 28 位地址，剩下最高 4 位由 pc+4 最高 4 位拼接上。

==> 停机指令

(16) halt

111111	00000000000000000000000000000000 (26 位)
--------	---

功能：停机；不改变 PC 的值，PC 保持不变。

实验要求：复位信号（reset）接开关 SW0，按键（单脉冲）接按键 BTNR。
需满足让开关 SW_in(SW15, SW14)控制七段数码管的显示内容，当 SW_in =

- 00 时，显示当前 PC 值：下条指令的 PC 值
- 01 时，显示 RS 寄存器地址：RS 寄存器数据
- 10 时，显示 RT 寄存器地址：RT 寄存器数据
- 11 时，显示 ALU 结果输出：DB 总线数据

指令执行采用单步（按键控制）执行方式，由开关（SW15, SW14）控制选择查看数码管上的相关信息，地址和数据。地址或数据的输出经一下模块代码转换到数码管上。

```
module Display_7SegLED(
    input [3:0] display_data,
    output reg [7:0] dispcode
);

always @ (display_data) begin
    case(display_data)
        4'b0000 : dispcode = 8'b1100_0000; //0; '0'-亮灯, '1'-熄灯
        4'b0001 : dispcode = 8'b1111_1001; //1
        4'b0010 : dispcode = 8'b1010_0100; //2
        4'b0011 : dispcode = 8'b1011_0000; //3
        4'b0100 : dispcode = 8'b1001_1001; //4
        4'b0101 : dispcode = 8'b1001_0010; //5
        4'b0110 : dispcode = 8'b1000_0010; //6
        4'b0111 : dispcode = 8'b1101_1000; //7
        4'b1000 : dispcode = 8'b1000_0000; //8
        4'b1001 : dispcode = 8'b1001_0000; //9
        4'b1010 : dispcode = 8'b1000_1000; //A
        4'b1011 : dispcode = 8'b1000_0011; //b
        4'b1100 : dispcode = 8'b1100_0110; //C
        4'b1101 : dispcode = 8'b1010_0001; //d
        4'b1110 : dispcode = 8'b1000_0110; //E
        4'b1111 : dispcode = 8'b1000_1110; //F
        default : dispcode = 8'b0000_0000; //不亮
    endcase
end
```

```
        endcase
    end
endmodule
```

三 实验原理

单周期 CPU 指的是一条指令的执行在一个时钟周期内完成，然后开始下一条指令的执行，即一条指令用一个时钟周期完成。电平从低到高变化的瞬间称为时钟上升沿，两个相邻时钟上升沿之间的时间间隔称为一个时钟周期。时钟周期一般也称振荡周期（如果晶振的输出没有经过分频就直接作为 CPU 的工作时钟，则时钟周期就等于振荡周期。若振荡周期经二分频后形成时钟脉冲信号作为 CPU 的工作时钟，这样，时钟周期就是振荡周期的两倍。）

CPU 在处理指令时，一般需要经过以下几个步骤：

(1) 取指令(IF)：根据程序计数器 PC 中的指令地址，从存储器中取出一条指令，同时，PC 根据指令字长度自动递增产生下一条指令所需要的指令地址，但遇到“地址转移”指令时，则控制器把“转移地址”送入 PC，当然得到的“地址”需要做些变换才送入 PC。

(2) 指令译码(ID)：对取指令操作中得到的指令进行分析并译码，确定这条指令需要完成的操作，从而产生相应的操作控制信号，用于驱动执行状态中的各种操作。

(3) 指令执行(EXE)：根据指令译码得到的操作控制信号，具体地执行指令动作，然后转移到结果写回状态。

(4) 存储器访问(MEM)：所有需要访问存储器的操作都将在这个步骤中执行，该步骤给出存储器的数据地址，把数据写入到存储器中数据地址所指定的存储单元或者从存储器中得到数据地址单元中的数据。

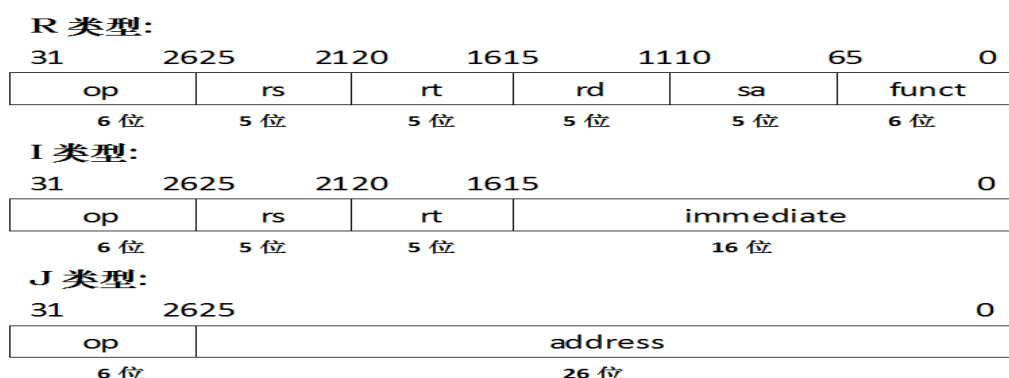
(5) 结果写回(WB)：指令执行的结果或者访问存储器中得到的数据写回相应的目的寄存器中。

单周期 CPU，是在一个时钟周期内完成这五个阶段的处理。



图 1 单周期 CPU 指令处理过程

MIPS 指令的三种格式：



其中，

- op:** 为操作码；
- rs:** 只读。为第 1 个源操作数寄存器，寄存器地址（编号）是 00000~11111，00~1F；
- rt:** 可读可写。为第 2 个源操作数寄存器，或目的操作数寄存器，寄存器地址（同上）；
- rd:** 只写。为目的操作数寄存器，寄存器地址（同上）；
- sa:** 为位移量（shift amt），移位指令用于指定移多少位；
- funct:** 为功能码，在寄存器类型指令中（R 类型）用来指定指令的功能与操作码配合使用；
- immediate:** 为 16 位立即数，用作无符号的逻辑操作数、有符号的算术操作数、数据加载（Load）/数据保存（Store）指令的数据地址字节偏移量和分支指令中相对程序计数器（PC）的有符号偏移量；
- address:** 为地址。

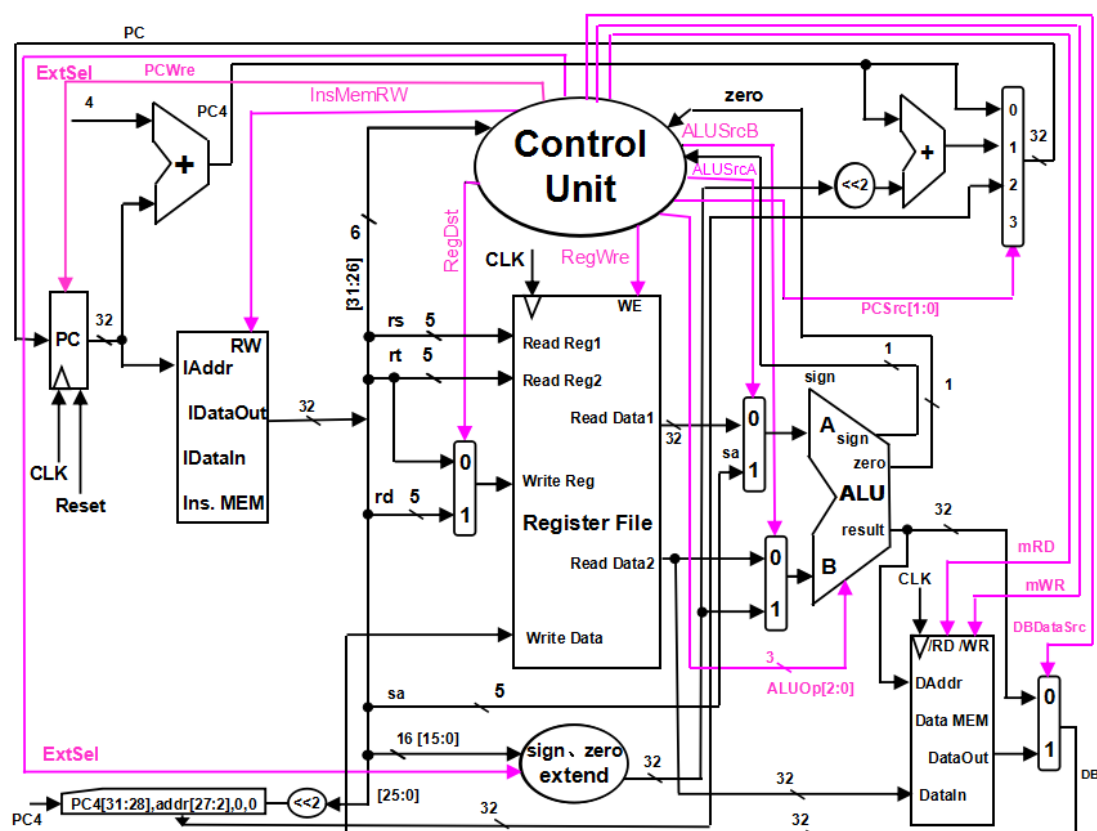


图 2 单周期 CPU 数据通路和控制线路图

图 2 是一个简单的基本上能够在单周期 CPU 上完成所要求设计的指令功能的数据通路和必要的控制线路图。其中指令和数据各存储在不同存储器中，即有指令存储器和数据存储器。访问存储器时，先给出内存地址，然后由读或写信号控制操作。对于寄存器组，先给出寄存器地址，读操作时不需要时钟信号，输出端就直接输出相应数据；而在写操作时，在 WE 使能信号为 1 时，在时钟边沿触发将数据写入寄存器。图中控制信号作用如表 1 所示，表 2 是 ALU 运算功能表。

表 1 控制信号的作用

控制信号名	状态“0”	状态“1”
Reset	初始化 PC 为 0	PC 接收新地址
PCWre	PC 不更改，相关指令：halt	PC 更改，相关指令：除指令 halt 外
ALUSrcA	来自寄存器堆 data1 输出，相关指令：add、sub、addiu、or、and、andi、ori、slti、beq、bne、blez、sw、lw	来自移位数 sa，同时，进行 (zero-extend)sa，即 $\{ \{27\{1'b0\}\}, sa \}$ ，相关指令：sll
ALUSrcB	来自寄存器堆 data2 输出，相关指令：add、sub、or、and、beq、bne、blez	来自 sign 或 zero 扩展的立即数，相关指令：addi、andi、ori、slti、sw、lw
DBDataSrc	来自 ALU 运算结果的输出，相关指令：add、addiu、sub、ori、or、and、andi、slti、sll	来自数据存储器（Data MEM）的输出，相关指令：lw
RegWre	无写寄存器组寄存器，相关指令：beq、bne、blez、sw、halt	寄存器组写使能，相关指令：add、addiu、sub、ori、or、and、andi、slti、sll、lw
InsMemRW	写指令存储器	读指令存储器 (Ins. Data)
mRD	输出高阻态	读数据存储器，相关指令：lw
mWR	无操作	写数据存储器，相关指令：sw
RegDst	写寄存器组寄存器的地址，来自 rt 字段，相关指令：addiu、andi、ori、slti、lw	写寄存器组寄存器的地址，来自 rd 字段，相关指令：add、sub、and、or、sll
ExtSel	(zero-extend)immediate (0 扩展)，相关指令：addiu、andi、ori	(sign-extend)immediate (符号扩展)，相关指令：slti、sw、lw、beq、bne、blez
PCSrc[1..0]	00: $pc < -pc + 4$ ，相关指令：add、addiu、sub、or、ori、and、andi、slti、sll、sw、lw、beq(zero=0)、bne(zero=1)、blez(sign=0&&zero=0)； 01: $pc < -pc + 4 + (\text{sign-extend})immediate$ ，相关指令：beq(zero=1)、bne(zero=0)、blez(sign=1 zero=1)； 10: $pc < -\{(pc+4)[31:28], \text{addr}[27:2], 2'b00\}$ ，相关指令：j；	

	11: 未用
ALUOp[2..0]	ALU 8 种运算功能选择(000-111)，看功能表

相关部件及引脚说明：（下面是实验要求，和我的变量名字不一样）

Instruction Memory: 指令存储器，

Iaddr, 指令存储器地址输入端口

IDataIn, 指令存储器数据输入端口（指令代码输入端口）

IDataOut, 指令存储器数据输出端口（指令代码输出端口）

RW, 指令存储器读写控制信号，为 0 写，为 1 读

Data Memory: 数据存储器，

Daddr, 数据存储器地址输入端口

DataIn, 数据存储器数据输入端口

DataOut, 数据存储器数据输出端口

mRD, 数据存储器读控制信号，为 0 读

mWR, 数据存储器写控制信号，为 0 写

Register File: 寄存器组

Read Reg1, rs 寄存器地址输入端口

Read Reg2, rt 寄存器地址输入端口

Write Reg, 将数据写入的寄存器端口，其地址来源 rt 或 rd 字段

Write Data, 写入寄存器的数据输入端口

Read Data1, rs 寄存器数据输出端口

Read Data2, rt 寄存器数据输出端口

WE, 写使能信号，为 1 时，在时钟边沿触发写入

ALU: 算术逻辑单元

result, ALU 运算结果

zero, 运算结果标志，结果为 0，则 zero=1；否则 zero=0

sign, 运算结果标志，结果最高位为 0，则 sign=0，正数；否则，sign=1，

负数

表 2 ALU 运算功能表

ALUOp[2..0]	功能	描述
000	$Y = A + B$	加
001	$Y = A - B$	减
010	$Y = B \ll A$	B 左移 A 位
011	$Y = A \vee B$	或
100	$Y = A \wedge B$	与
101	$Y = (A \leq B) ? 1 : 0$	比较 A 与 B 不带符号
110	$Y = (((\text{rega} < \text{regb}) \quad \&\& \\ (\text{rega}[31] == \text{regb}[31]) \quad)) \\ ((\text{rega}[31] == 1 \quad \&\& \\ \text{regb}[31] == 0))) ? 1 : 0$	比较 A 与 B 带符号
111	$Y = A \oplus B$	异或

四 实验设备

PC 机一台，BASYS 3 实验板一块，Xilinx Vivado 开发软件一套。

五 实验过程和结果

单周期 CPU 整体数据通路复杂，控制信号繁多。在数据通路上，我按照理论课所学把单周期 CPU 拆分成可以相对独立运行的若干个模块，有 ALU、RegFile、DataMemory、InstructionMemory 等模块。控制信号的生成上按照老师要求分为主控和辅控两个模块，主控处理 opcode 信号，辅控处理 aluop（内部连线）和 funct 信号。最后在顶层模块将所有小模块综合，实现了可运行 16 条指令的单周期 CPU。

以下是各个模块的设计

1.控制信号的生成

控制信号的生成分成两部分，主控 ctr 处理 opcode，局控 aluctr 处理 funct。需要先列出控制信号和指令的关系：

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	Instruction op	funct	PCWre	ALUSrcA	ALUSrcB	DBDataSrc	RegWre	InsMemRW	mRD	mWR	RegDst	ExtS	PCSrc[1:0]	ALUOp[2:0]	
2	add	/000000	/100000	1	0	0	0	1	1	0	0	1 x	/00	/000	
3	sub	/000000	/100010	1	0	0	0	1	1	0	0	1 x	/00	/001	
4	addiu	/001001	x	1	0	1	0	1	1	0	0	0	1 /00	/000	
5	andi	/001100	x	1	0	1	0	1	1	0	0	0	0 /00	/100	
6	and	/000000	/100100	1	0	0	0	1	1	0	0	1	/00	/100	
7	ori	/001101	x	1	0	1	0	1	1	0	0	0	0 /00	/011	
8	or	/000000	/100101	1	0	0	0	1	1	0	0	1 x	/00	/011	
9	sll	/000000	/000000	1	1	0	0	1	1	0	0	1 x	/00	/010	
10	slli	/001010	x	1	0	1	0	1	1	0	0	0	1 /00	/110	
11	sw	/101011	x	1	0	1	0	0	1	0	1 x		1 /00	/000	
12	lw	/100011	x	1	0	1	1	1	1	1	0	0	1 /00	/000	
13	beq	/000100	x	1	0	0 x		0	1	0	0 x		1 /01	/001	
14	bne	/000101	x	1	0	0 x		0	1	0	0 x		1 /01	/001	
15	blez	/000110	x	1	0	0 x		0	1	0	0 x		1 /01	/101	
16	j	/000010	x	1	0 x	x		0	1	0	0 x	x	/10	x	
17	halt	/111111	x	0	0 x	x		0	1	0	0 x	x	x	x	

1) ctr 为负责主要控制信号的模块。

输入：

- opCode：从 InstructionMemory 读取的指令 op 码
- zero：从 ALU 读取的数据，为 1 则 ALU 运算结果 result 为 0，bne、beq 指令需要通过 result 是否为零判断是否跳转
- sign：从 ALU 读取的数据，为 1 为负数，blez 指令通过 result 的符号判断是否跳转

输出：

- PCWre,ALUsrcB,DBDataSrc,RegWre,InsMemRW,mRD,mWR,RegDst,ExtSel,PCSrc：控制信号
- aluop：输出给 aluctr 的一个内部连线

模块代码：

```
module ctr(//主控，产生控制信号和 aluctr
    input [5:0] opCode,
    input zero,
    input sign,
    output reg PCWre,
    output reg ALUsrcB,
    output reg DBDataSrc,
    output reg RegWre,
    output reg InsMemRW,
    output reg mRD,
    output reg mWR,
    output reg RegDst,
    output reg ExtSel, //符号扩展方式，1 为 sign-extend, 0 为 zero-
extend
    output reg [1:0]PCSrc,

    output reg [3:0] aluop // 经过 ALU 控制译码决定 ALU 功能,为内部
信号

);
```

```

initial begin
    PCWre=1;  ALUsrcB=1;  DBDataSrc=0;  RegWre=1;  InsMemRW=1;
    mRD=0;  mWR=0;  RegDst=0;  ExtSel=1;  PCSrc[1:0]=2'b00;
    aluop = 4'b0000;
end

always@(*) begin
    // 操作码改变时改变控制信号
    case(opCode)
        6'b000010: begin
            PCWre=1;  ALUsrcB=1'bz;  DBDataSrc=1'bz;  RegWre=0;
InsMemRW=1;
            mRD=0;  mWR=0;  RegDst=1'bz;  ExtSel=1'bz;  PCSrc=2'b10;
            aluop = 4'b0000;
            end // 'J 型' 指令操作码: 000010, 无需 ALU

            6'b000000: begin
                PCWre=1; ALUsrcB=0;  DBDataSrc=0;  RegWre=1; InsMemRW=1;
                mRD=0;  mWR=0;  RegDst=1;  ExtSel=1'bz;  PCSrc[1:0]=2'b00; //
ALUOp[2:0]=3'bz;
                aluop = 4'b1111;
                end // 'R 型' 指令操作码: 000000,sll 除外, alusrc=1;

                // 'I' 型指令操作码
                6'b100011: begin
                    PCWre=1;  ALUsrcB=1;  DBDataSrc=1;  RegWre=1; InsMemRW=1;
                    mRD=1;  mWR=0;  RegDst=0;  ExtSel=1;  PCSrc[1:0]=2'b00;
                    aluop = 4'b0000;
                    end // 'lw' 指令操作码: 100011

                    6'b101011: begin
                        PCWre=1; ALUsrcB=1;  DBDataSrc=0;  RegWre=0; InsMemRW=1;
                        mRD=0;  mWR=1;  RegDst=1'bz;  ExtSel=1;  PCSrc[1:0]=2'b00;
                        aluop = 4'b0000;
                        end // 'sw' 指令操作码: 101011

                        6'b000100: begin
                            PCWre=1; ALUsrcB=0;  DBDataSrc=1'bz;  RegWre=0; InsMemRW=1;
                            mRD=0;  mWR=0;  RegDst=1'bz;  ExtSel=1;
                            PCSrc[1:0]=zero==1? 2'b01:2'b00;
                            aluop = 4'b0001;
                            end // 'beq' 指令操作码: 000100

                            6'b000101: begin

```

```

PCWre=1; ALUsrcB=0; DBDataSrc=1'bz; RegWre=0; InsMemRW=1;
mRD=0; mWR=0; RegDst=1'bz; ExtSel=1;
PCSrc[1:0]=zero==0? 2'b01:2'b00;
aluop = 4'b0110;
end // 'bne' 指令操作码: 000101

```

```

6'b000110: begin
PCWre=1; ALUsrcB=0; DBDataSrc=1'bz; RegWre=0; InsMemRW=1;
mRD=0; mWR=0; RegDst=1'bz; ExtSel=1;
PCSrc[1:0]=(zero==1||sign==1)? 2'b01:2'b00;
aluop = 4'b0101;
end // 'blez' 指令操作码: 000101

```

```

6'b001001: begin
PCWre=1; ALUsrcB=1; DBDataSrc=0; RegWre=1; InsMemRW=1;
mRD=0; mWR=0; RegDst=0; ExtSel=1; PCSrc[1:0]=2'b00;
aluop = 4'b0000;
end // 'addiu' 指令操作码: 001000

```

```

6'b001100: begin
PCWre=1; ALUsrcB=1; DBDataSrc=0; RegWre=1; InsMemRW=1;
mRD=0; mWR=0; RegDst=0; ExtSel=0; PCSrc[1:0]=2'b00;
aluop = 4'b0100;
end // 'andi' 指令操作码: 001100

```

```

6'b001101: begin
PCWre=1; ALUsrcB=1; DBDataSrc=0; RegWre=1; InsMemRW=1;
mRD=0; mWR=0; RegDst=0; ExtSel=0; PCSrc[1:0]=2'b00;
aluop = 4'b0010;
end // 'ori' 指令操作码: 001101

```

```

6'b001010: begin
PCWre=1; ALUsrcB=1; DBDataSrc=0; RegWre=1; InsMemRW=1;
mRD=0; mWR=0; RegDst=0; ExtSel=1; PCSrc[1:0]=2'b00;
aluop = 4'b0011;
end // 'slti' 指令操作码: 001010

```

```

6'b111111: begin
//pc 保持不变
PCWre=0; ALUsrcB=1'bz; DBDataSrc=1'bz; RegWre=1'bz;
InsMemRW=1'bz;
mRD=1'bz; mWR=1'bz; RegDst=1'bz; ExtSel=1'bz;
PCSrc[1:0]=2'bz;

```

```

        end // 'halt' 指令操作码: 111111

        default: begin
            PCWre=1'b1; ALUsrcB=1'b1; DBDataSrc=1'b1; RegWre=1'b1;
InsMemRW=1'bz;
            mRD=1'bz; mWR=1'bz; RegDst=1'bz; ExtSel=1'bz;
PCSrc[1:0]=2'bz;
            end // 默认设置
        endcase end
    endmodule

```

测试代码:

```

module test_ctr;

    // 输入信号
    reg [5:0] opCode;
    reg zero;
    reg sign;

    // 输出信号
    wire PCWre;
    wire ALUsrcB;
    wire DBDataSrc;
    wire RegWre;
    wire InsMemRW;
    wire mRD;
    wire mWR;
    wire RegDst;
    wire ExtSel;
    wire [1:0] PCSrc;
    wire [3:0] aluop;

    // 实例化 ctr 模块
    ctr uut (
        .opCode(opCode),
        .zero(zero),
        .sign(sign),
        .PCWre(PCWre),
        .ALUsrcB(ALUsrcB),
        .DBDataSrc(DBDataSrc),
        .RegWre(RegWre),
        .InsMemRW(InsMemRW),
        .mRD(mRD),

```

```

        .mWR(mWR),
        .RegDst(RegDst),
        .ExtSel(ExtSel),
        .PCSrc(PCSrc),
        .aluop(aluop)
    );

// 测试过程
initial begin
    // 初始化信号
    opCode = 6'b000000; zero = 0; sign = 0;
    #10; // 等待 10 个时间单位, 模拟时钟周期

    // 打印输出
    $display("Test Case 1: opCode = %b (R-type, 000000)");
    $display("PCWre = %b, ALUsrcB = %b, DBDataSrc = %b, RegWre
= %b, InsMemRW = %b",
            PCWre, ALUsrcB, DBDataSrc, RegWre, InsMemRW);
    $display("mRD = %b, mWR = %b, RegDst = %b, ExtSel = %b, PCSrc
= %b, aluop = %b",
            mRD, mWR, RegDst, ExtSel, PCSrc, aluop);
    #10;

    // 'lw' 指令 (100011)
    opCode = 6'b100011;
    zero = 0; sign = 0;
    #10;
    $display("Test Case 2: opCode = %b (lw)");
    $display("PCWre = %b, ALUsrcB = %b, DBDataSrc = %b, RegWre
= %b, InsMemRW = %b",
            PCWre, ALUsrcB, DBDataSrc, RegWre, InsMemRW);
    $display("mRD = %b, mWR = %b, RegDst = %b, ExtSel = %b, PCSrc
= %b, aluop = %b",
            mRD, mWR, RegDst, ExtSel, PCSrc, aluop);
    #10;

    // 'sw' 指令 (101011)
    opCode = 6'b101011;
    zero = 0; sign = 0;
    #10;
    $display("Test Case 3: opCode = %b (sw)");
    $display("PCWre = %b, ALUsrcB = %b, DBDataSrc = %b, RegWre
= %b, InsMemRW = %b",
            PCWre, ALUsrcB, DBDataSrc, RegWre, InsMemRW);
    $display("mRD = %b, mWR = %b, RegDst = %b, ExtSel = %b, PCSrc

```

```

= %b, aluop = %b",
    mRD, mWR, RegDst, ExtSel, PCSrc, aluop);
#10;

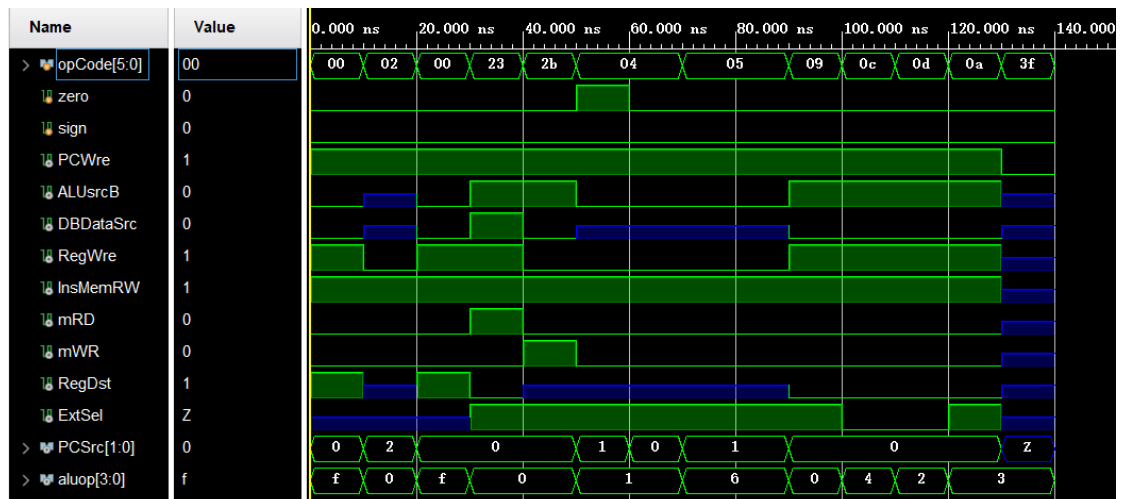
// 'beq' 指令 (000100)
opCode = 6'b000100; zero = 1; sign = 0;
#10;
$display("Test Case 4: opCode = %b (beq, zero = %b)", opCode,
zero);
$display("PCWre = %b, ALUsrcB = %b, DBDataSrc = %b, RegWre
= %b, InsMemRW = %b",
    PCWre, ALUsrcB, DBDataSrc, RegWre, InsMemRW);
$display("mRD = %b, mWR = %b, RegDst = %b, ExtSel = %b, PCSrc
= %b, aluop = %b",
    mRD, mWR, RegDst, ExtSel, PCSrc, aluop);
#10;

// 'bne' 指令 (000101)
opCode = 6'b000101; zero = 0; sign = 0;
#10;
$display("Test Case 5: opCode = %b (bne, zero = %b)", opCode,
zero);
$display("PCWre = %b, ALUsrcB = %b, DBDataSrc = %b, RegWre
= %b, InsMemRW = %b",
    PCWre, ALUsrcB, DBDataSrc, RegWre, InsMemRW);
$display("mRD = %b, mWR = %b, RegDst = %b, ExtSel = %b, PCSrc
= %b, aluop = %b",
    mRD, mWR, RegDst, ExtSel, PCSrc, aluop);
#10;

$finish;

```

输出波形:



2) aluctr 为主要负责 ALUOp 的模块

输入:

- aluop: 来自 ctr 的内部连线;
- funct: 来自指令内存的 funct 码;

输出:

- ALUOp: 控制 ALU 做何种运算的信号;
- ALUSrcA: 因为只有 sll 指令的 ALUSrcA 为 1, 但是 sll 为 R 型指令, 只能放在 aluctr 通过 funct 来处理。

模块代码:

```
module aluctr(
    input [3:0] aluop,
    input [5:0] funct,
    output reg [2:0] ALUOp,
    output reg ALUSrcA
);

    initial begin
        ALUOp = 3'b000;
        ALUSrcA = 0;
    end

    always @(aluop or funct) begin
        casez({aluop, funct})
            // i 型指令
            10'b0000?????: begin
                ALUOp = 3'b000; // add
                ALUSrcA = 0;
            end
            // lw, sw, addiu
            10'b0001?????: begin
                ALUOp = 3'b001; // beq
                ALUSrcA = 0;
            end
            // bne, beq
            10'b0101?????: begin
```



```

        ALUOp = 3'b101; // beq
        ALUSrcA = 0;
    end
    // blez
    10'b0110?????: begin
        ALUOp = 3'b001; // bne
        ALUSrcA = 0;
    end
    10'b0010?????: begin
        ALUOp = 3'b011; // ori
        ALUSrcA = 0;
    end
    10'b0011?????: begin
        ALUOp = 3'b110; // slti
        ALUSrcA = 0;
    end
    10'b0100?????: begin
        ALUOp = 3'b100; // andi
        ALUSrcA = 0;
    end

    // r 型指令
    10'b1111100000: begin
        ALUOp = 3'b000; // add
        ALUSrcA = 0;
    end
    10'b1111100010: begin
        ALUOp = 3'b001; // sub
        ALUSrcA = 0;
    end
    10'b1111100100: begin
        ALUOp = 3'b100; // and
        ALUSrcA = 0;
    end
    10'b1111100101: begin
        ALUOp = 3'b011; // or
        ALUSrcA = 0;
    end
    10'b1111000000: begin
        ALUOp = 3'b010; // sll
        ALUSrcA = 1;
    end

    default: begin

```

```

        ALUOp = 3'b000;
        ALUSrcA = 0;
    end
endcase
end

endmodule

```

仿真代码:

```

module tb_aluctr;

    // 声明输入和输出
    reg [3:0] aluop;
    reg [5:0] funct;
    wire [2:0] ALUOp;
    wire ALUSrcA;

    // 实例化 aluctr 模块
    aluctr uut (
        .aluop(aluop),
        .funct(funct),
        .ALUOp(ALUOp),
        .ALUSrcA(ALUSrcA)
    );

    // 仿真过程
    initial begin
        // 初始化信号
        aluop = 4'b0000; funct = 6'b000000; // 第一个测试, i 型指令
        #10; // 延时 10 时间单位

        aluop = 4'b0001; funct = 6'b000000; // 第二个测试, lw, sw, addiu
        #10;

        aluop = 4'b0101; funct = 6'b000000; // 第三个测试, bne, beq
        #10;

        aluop = 4'b0110; funct = 6'b000000; // 第四个测试, blez
        #10;

        aluop = 4'b0010; funct = 6'b000000; // 第五个测试, ori
        #10;
    end
endmodule

```

```

aluop = 4'b0011; funct = 6'b000000; // 第六个测试, slti
#10;

aluop = 4'b0100; funct = 6'b000000; // 第七个测试, andi
#10;

aluop = 4'b1111; funct = 6'b000000; // 第八个测试, r 型指令 (add)
#10;

aluop = 4'b1111; funct = 6'b000010; // 第九个测试, r 型指令 (sub)
#10;

aluop = 4'b1111; funct = 6'b000100; // 第十个测试, r 型指令 (and)
#10;

aluop = 4'b1111; funct = 6'b000101; // 第十一个测试, r 型指令 (or)
#10;

aluop = 4'b1111; funct = 6'b000000; // 第十二个测试, r 型指令 (sll)
#10;

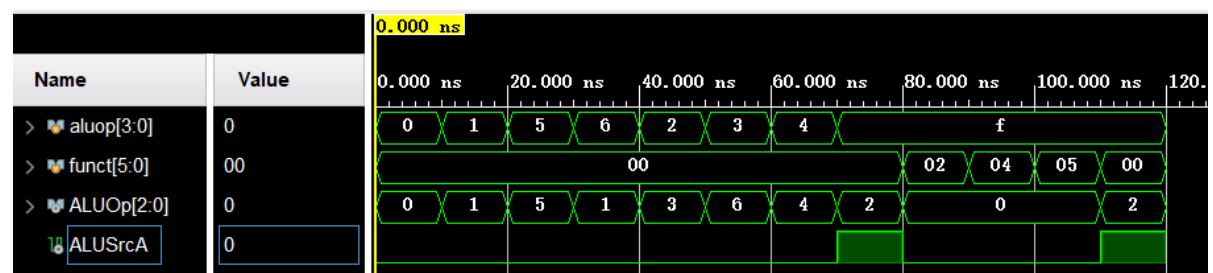
// 停止仿真
$finish;
end

// 观察信号变化
initial begin
    $monitor("Time = %0t, aluop = %b, funct = %b, ALUOp = %b, ALUSrcA
= %b", $time, aluop, funct, ALUOp, ALUSrcA);
end

endmodule

```

仿真波形:



2. ALU

ALU 是 CPU 中的负责逻辑运算的模块。该模块接受 5 个输入：

- ra : 从 rs 寄存器读取的数据
- rb: 从 rt 寄存器读取的数据
- immediateExt: 从符号拓展单元读取的数据
- shamt: 直接从指令中解码得到的 Shame 字段
- ALUOp : 从控制单元读取的 ALU 控制信号，根据这个决定 ALU 的运算功能
- ALUSrcA : 决定进行计算的数据是 ReadData1 还是 Sa
- ALUSrcB : 决定进行计算的数据是 ReadData2 还是 Ext

该模块输出三个信号：

- zero : 输出为 0 时为 1，否则为 0
- sign: 输出为 1 为负数
- result : 进行运算后的结果

模块代码：

```
module ALU(//..  
    input [31:0]ra,  
    input [31:0]rb,  
    input [4:0]shamt,  
    input [31:0]immediateExt,  
    input [2:0]ALUOp,  
    input ALUSrcA,//选择器，控制信号  
    input ALUSrcB,  
  
    output reg[31:0]result,  
    output sign,  
    output zero  
);
```

```

wire [31:0]inputa;
wire [31:0]inputb;

assign inputa=ALUSrcA? {{27{1'b0}},shamt}:ra;
assign inputb=ALUSrcB? immediateExt : rb;
assign sign=(inputa<inputb) ? 1:0;//非负数为 0， 负数 1
assign zero=(result==32'b0)? 1:0;

always@(*)begin
    case(ALUOp)
        3'b000://+
            result=inputa+inputb;
        3'b001://-
            result=inputa-inputb;
        3'b010://<<
            result=inputb<<inputa;
        3'b011:
            result=inputa|inputb;
        3'b100:
            result=inputa&inputb;
        3'b101:
            result=inputa<=inputb?1:0;
        3'b110:
            result=(((inputa< inputb) && ( inputa[31] ==
inputb[31] )) || ( ( inputa[31] ==1 && inputb[31] == 0))) ? 1:0;
        3'b111:
            result=inputa^^inputb;
        default:
            result = 32'h0000;
    endcase
end
endmodule

```

测试代码：

```

module tb_ALU;

    // 输入信号
    reg [31:0] ra;
    reg [31:0] rb;
    reg [4:0] shamt;
    reg [31:0] immediateExt;
    reg [2:0] ALUOp;

```

```

reg ALUSrcA;
reg ALUSrcB;

// 输出信号
wire [31:0] result;
wire sign;
wire zero;

// 实例化 ALU 模块
ALU uut (
    .ra(ra),
    .rb(rb),
    .shamt(shamt),
    .immediateExt(immediateExt),
    .ALUOp(ALUOp),
    .ALUSrcA(ALUSrcA),
    .ALUSrcB(ALUSrcB),
    .result(result),
    .sign(sign),
    .zero(zero)
);

// 仿真过程
initial begin
    // 测试初始化
    ra = 32'b00000000000000000000000000000001; // 1
    rb = 32'b00000000000000000000000000000010; // 2
    shamt = 5'b00001; // 左移的位数
    immediateExt = 32'b00000000000000000000000000000100; // 4

    ALUSrcA = 0; ALUSrcB = 0; ALUOp = 3'b000; // 加法
    #10;

    ALUOp = 3'b001; // 减法
    #10;

    ALUOp = 3'b010; // 左移
    #10;

    ALUOp = 3'b011; // OR
    #10;

    ALUOp = 3'b100; // AND
    #10;

```

```

    ALUOp = 3'b101; // 小于等于比较
    #10;

    ALUOp = 3'b110; // 小于比较（有符号数比较）
    #10;

    ALUOp = 3'b111; // XNOR
    #10;

    // 测试 ALUSrcA 和 ALUSrcB 控制信号
    ALUSrcA = 1; ALUSrcB = 1; ALUOp = 3'b000; // 加法，使用 shamt 和
immediateExt
    #10;

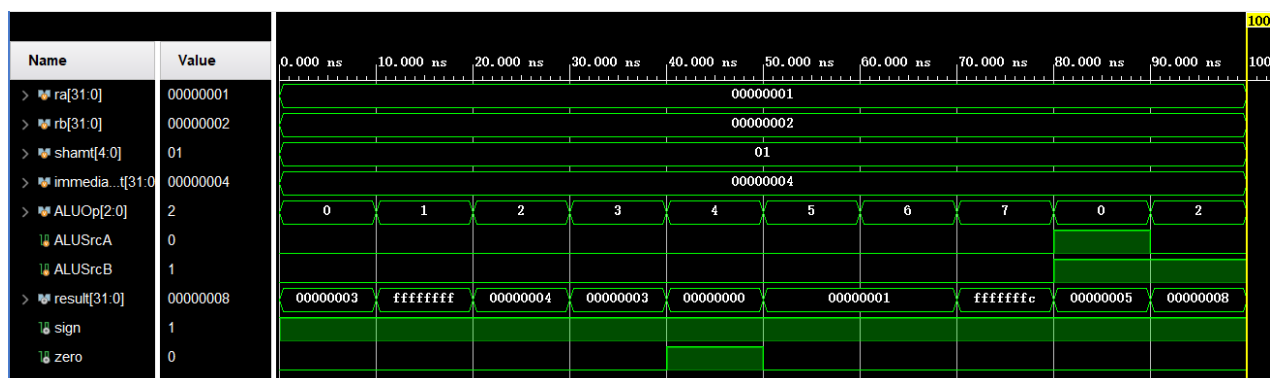
    ALUSrcA = 0; ALUSrcB = 1; ALUOp = 3'b010; // 左移，使用
immediateExt
    #10;

    // 停止仿真
    $finish;
end

// 观察信号
initial begin
    $monitor("Time = %0t, ra = %d, rb = %d, shamt = %b, immediateExt
= %d, ALUOp = %b, result = %d, sign = %b, zero = %b",
            $time, ra, rb, shamt, immediateExt, ALUOp, result, sign,
zero);
end
endmodule

```

输出波形：



3. DataMemory

DataMemory 用于将数据存储到内存中和从内存中读取数据，该模块接受 4 个输入：

- CLK : 时钟信号，只允许在时钟下降沿进行写操作
- raAddIm: 读取或写数据的地址
- datain : 要进行写操作的数据
- mRD : 为 0 进行读操作
- mWR : 为 0 进行写操作

该模块输出数据 32 位 DataOut，允许读时输出从内存读出的数据，不允许读时输出高阻态。

模块代码：

```
module DataMemory (//mRD,mWR
    input CLK,
    input mWR,//写使能 0 写
    input mRD,//读 0
    input wire [31:0]raAddIm,//lw,sw 指令中输入的为 ra+im,还没有乘以四,
    输入到 address,cpu 里应该是 result 吧
    input wire [31:0]datain,//sw 时使用，输入为要写入的寄存器的地址，应
    该是 rs，所以接口应该是 ra，好像那个实验报告有问题

    output wire[31:0]dataout
);
//内存体
reg [7:0]memory[0:127];//按字节编址，128 个字节
wire[31:0]address;

assign address=(raAddIm<<2); //*4

always@(negedge CLK) begin
    if(mWR==0) begin
        memory[address]<=datain[31:24];
        memory[address+1]<=datain[23:16];
        memory[address+2]<=datain[15:8];
    end
end
```



```

        memory[address+3]<=datain[7:0];
    end
end

assign dataout[7:0] = mRD==0 ? memory[address+3]:8'bz;
assign dataout[15:8]= mRD==0 ? memory[address+2]:8'bz;
assign dataout[23:16]= mRD==0 ? memory[address+1]:8'bz;
assign dataout[31:24]= mRD==0 ? memory[address]:8'bz;
endmodule

```

仿真代码：

```

module tb_DataMemory;
    reg CLK;
    reg mWR;
    reg mRD;
    reg [31:0] raAddIm;    // 地址输入
    reg [31:0] datain;     // 写入数据
    wire [31:0] dataout;   // 读取数据

    // 实例化 DataMemory 模块
    DataMemory uut (
        .CLK(CLK),
        .mWR(mWR),
        .mRD(mRD),
        .raAddIm(raAddIm),
        .datain(datain),
        .dataout(dataout)
    );

    // 时钟生成
    always begin
        CLK = ~CLK;
        #5;    // 时钟周期 10ns
    end

    initial begin
        // 初始化信号
        CLK = 0;
        mWR = 1;    // 1 表示不写入
        mRD = 1;    // 1 表示不读取
        raAddIm = 32'b0;
        datain = 32'b0;
    end
endmodule

```

```

// 等待系统稳定
#10;

// 测试写操作
// 写入数据 0x12345678 到地址 4
raAddIm = 32'd1; // 地址=ra+im, 这里设置为 1 表示地址 4
datain = 32'h12345678;
mWR = 0; // 使能写入
mRD = 1; // 不读取
#10; // 等待一个时钟周期

// 测试读操作
// 读取地址 4 的数据
raAddIm = 32'd1; // 地址=ra+im, 这里设置为 1 表示地址 4
mWR = 1; // 不写入
mRD = 0; // 使能读取
#10; // 等待一个时钟周期

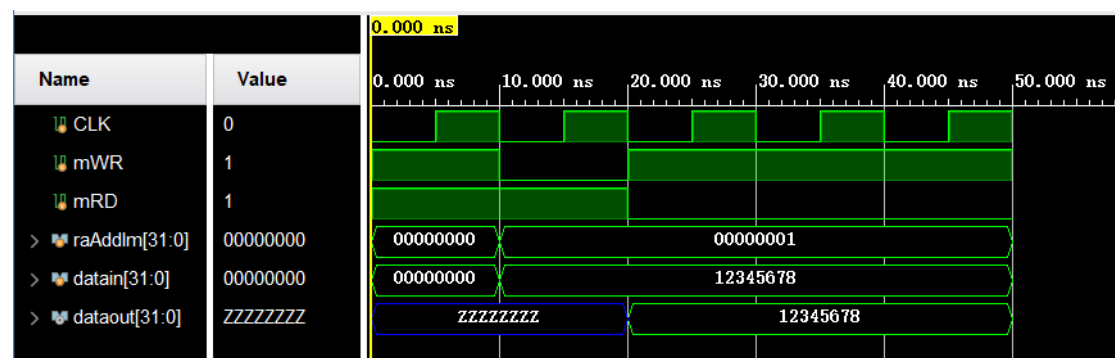
// 打印输出
$display("Read data: %h", dataout);

// 结束仿真
#20;
$finish;

end
endmodule

```

仿真波形：



4. InstructionMemory

指令存储器模块接受来自 PC 的地址并从存储器中取出指令并进行解码，然后输出解码后的字段。该模块接受 3 个输入：

- PC4 : 当前 PC 的前 4 位, 用于补全跳转指令 PC 的前四位
- IAddr : 指令的地址
- RW : 为 0 时写, 为 1 时读指令 (在该实验中不需要写指令, 所以 RW 恒为 1)

输出为:

- op : 输出到控制单元决定控制信号的输出
- rs : rs 寄存器地址
- rt : rt 寄存器地址
- rd : rd 寄存器地址
- Immediate : 立即数
- Sa : Shamt 位移字段
- JumpPC : 跳转 PC 的地址

模块代码:

```
module InstructionMemory(//InsMemRW
    input [3:0]pc4,//这个不就是指令地址吗
    input InsMemRW,//0 写 1 读,本来是 rw

    input [31:0]currentPC,//pc 来的吗

    output [5:0]op,
    output [4:0]rs,
    output [4:0]rt,
    output [4:0]rd,
    output [4:0]shamt,
    output [15:0]immediate,
    output [5:0]funct,
    output [31:0]jumppc
    //output [31:0]dataout
);
```

```

    reg [7:0]mem[0:127];
    reg [31:0] instruction;//指令

    assign op=instruction[31:26];
    assign rs=instruction[25:21];
    assign rt=instruction[20:16];
    assign rd=instruction[15:11];
    assign immediate=instruction[15:0];
    assign shamt=instruction[10:6];
    assign funct=instruction[5:0];
    assign jumppc={{pc4},{instruction[25:0]},{2'b00}};

    // 读取指令的过程
    initial begin
        $readmemb("C:/Users/zhang/Desktop/Instructions.txt", mem);//从文件中读
        取指令集
        // dataout = 0;//指令初始化
    end

    always@(*)begin
        if(InsMemRW==1)begin
            instruction[7:0]=mem[currentPC+3];
            instruction[15:8]=mem[currentPC+2];
            instruction[23:16]=mem[currentPC+1];
            instruction[31:24]=mem[currentPC];
        end
    end
endmodule

```

仿真代码:

```

`timescale 1ns / 1ps

module Testbench;

    // 变量声明
    reg [3:0] pc4;           // 测试时给定的 pc4
    reg InsMemRW;           // 读写使能信号
    reg [31:0] currentPC;    // 当前的 PC
    wire [5:0] op;           // 操作码
    wire [4:0] rs, rt, rd, shamt;
    wire [15:0] immediate;

```

```

wire [5:0] funct;
wire [31:0] jumppc;          // 跳转地址
// wire [31:0] dataout;      // 如果需要的话

// 实例化被测试模块
InstructionMemory uut (
    .pc4(pc4),
    .InsMemRW(InsMemRW),
    .currentPC(currentPC),
    .op(op),
    .rs(rs),
    .rt(rt),
    .rd(rd),
    .shamt(shamt),
    .immediate(immediate),
    .funct(funct),
    .jumppc(jumppc)
);

// 初始化信号
initial begin
    $display("开始仿真...");

    // 初始化
    pc4 = 4'b0000;          // 初始地址
    InsMemRW = 1'b1;        // 读操作
    currentPC = 32'h00000004; // 假设初始 PC 是 4

    // 读取文件并初始化内存
    $readmemb("C:/Users/zhang/Desktop/Instructions.txt", uut.mem);

    // 打印仿真信息
    $monitor("At time %t, currentPC = %h, instruction = %h, op = %h, rs
= %h, rt = %h, rd = %h, immediate = %h, jumppc = %h",
            $time, currentPC, uut.instruction, op, rs, rt, rd,
            immediate, jumppc);

    // 测试过程
    #10 currentPC = 32'h00000008; // 变更 PC 值
    #10 currentPC = 32'h0000000C; // 变更 PC 值
    #10 currentPC = 32'h00000010; // 变更 PC 值

    // 完成测试，结束仿真
    #10 $finish;

```

```

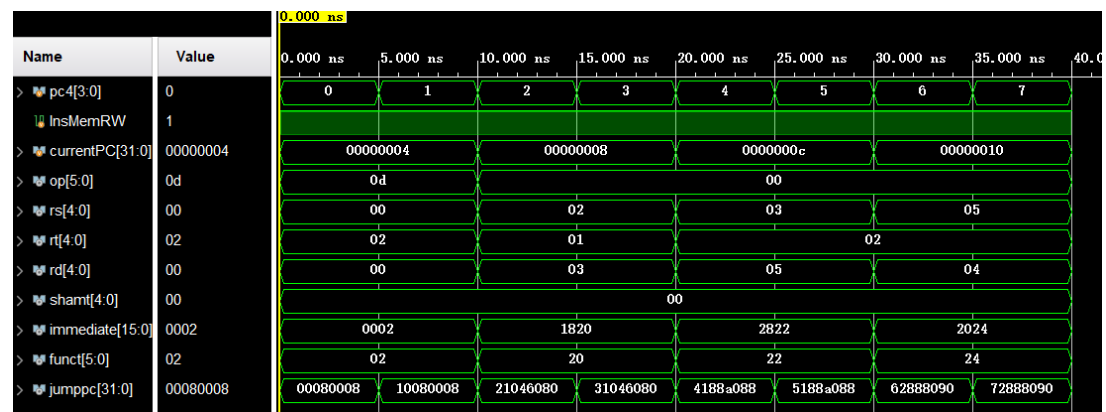
end

// 生成时钟信号，用于模拟时序
always begin
    #5 pc4 = pc4 + 1;
end

endmodule

```

仿真波形：



5. PC

PC 模块接受一个控制信号，决定将 PC+4 还是 PC+4+Immedate 还是 PC + 4 + JumpPC 送入到指令存储器进行指令读取。该模块有 6 个输入：

- CLK : 时钟信号，只允许在时钟的下降沿输出 PC
- Reset : PC 置零信号
- PCWre : 为 1 时才允许改变地址
- PCSrc : 决定输出的是何种 PC
- immediate : 跳转指令位移
- jumppc : 无条件跳转指令地址

有 3 个输出：

- currentPC: 输出指令的地址到指令存储器取出指令
- nextpc : 下一条指令的地址, 为 7 段数码管显示用
- pc4 : 无条件跳转指令高 4 位填补用

模块代码:

```
module pc( // PCWre, PCSrc, Reset 我没写
    input [1:0] PCSrc,          // 输出选择何种 pc, x1 bra, 10 jump, 00 pc+4
    input CLK,
    input PCWre,
    input signed [15:0] immediate,
    input [31:0] jumppc,
    input Reset,

    output reg signed [31:0] currentPC, // 当前 pc
    output [31:0] nextpc,
    output [3:0] pc4
);
    // 计算下一个 PC 值
    assign nextpc = PCSrc[0] ? (currentPC + (immediate << 2)) :
        (PCSrc[1] ? jumppc : currentPC + 4);
    assign pc4 = currentPC[31:28];

    always @(negedge CLK or negedge Reset) begin
        if (Reset == 0)
            currentPC <= 0;
        else if (PCWre == 1) begin // pc 可写
            if (PCSrc == 2'b00) // pc+4
                currentPC <= currentPC + 4;
            else if (PCSrc == 2'b01) // branch
                currentPC <= currentPC + (immediate << 2) + 4;
            else if (PCSrc == 2'b10) // jump
                currentPC <= jumppc;
        end
    end
endmodule
```

仿真代码:

```
module pc_tb;
    // 声明测试平台的信号
    reg [1:0] PCSrc;          // 输入选择信号
```

```

reg CLK;                // 时钟信号
reg PCWre;              // PC 是否可写
reg signed [15:0] immediate; // 偏移量
reg [31:0] jumppc;      // 跳转地址
reg Reset;              // 复位信号

// 输出信号
wire signed [31:0] currentPC; // 当前 PC 值
wire [31:0] nextpc;          // 下一个 PC 值
wire [3:0] pc4;              // PC 的高 4 位

// 实例化 pc 模块
pc uut (
    .PCSrc(PCSrc),
    .CLK(CLK),
    .PCWre(PCWre),
    .immediate(immediate),
    .jumppc(jumppc),
    .Reset(Reset),
    .currentPC(currentPC),
    .nextpc(nextpc),
    .pc4(pc4)
);

// 时钟生成（每 5ns 反转一次时钟）
always #5 CLK = ~CLK;

// 仿真过程
initial begin
    // 初始化信号
    CLK = 0;
    Reset = 0;
    PCWre = 0;
    PCSrc = 2'b00;
    immediate = 16'b0;
    jumppc = 32'b0;

    // 等待 10ns，拉低 Reset 信号，开始仿真
    #10;
    Reset = 1;    // 解除复位

    // 测试场景 1：进行 PC+4 操作
    #10;
    PCWre = 1;

```



```

PCSrc = 2'b00; // PC+4
immediate = 16'b0;
jumppc = 32'b0;
// 等待时钟上升沿
#10;

// 测试场景 2: 进行 Branch 操作
#10;
PCWre = 1;
PCSrc = 2'b01; // Branch
immediate = 16'd4; // 偏移量 4
jumppc = 32'b0;
// 等待时钟上升沿
#10;

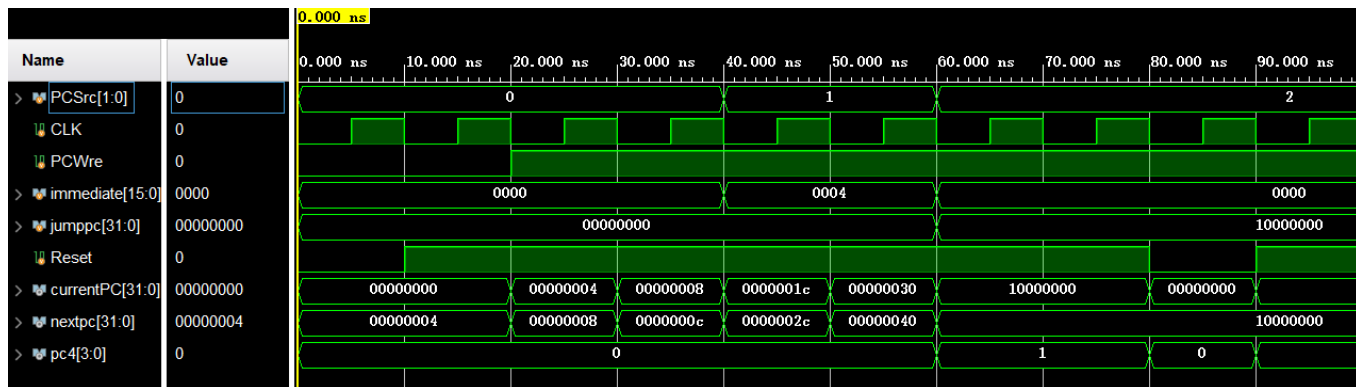
// 测试场景 3: 进行 Jump 操作
#10;
PCWre = 1;
PCSrc = 2'b10; // Jump
immediate = 16'b0;
jumppc = 32'h1000_0000; // 跳转到 0x1000_0000
// 等待时钟上升沿
#10;

// 测试场景 4: 复位测试
#10;
Reset = 0; // 拉低复位信号, 复位 PC
// 等待时钟上升沿
#10;
Reset = 1; // 解除复位信号
end

// 输出当前 PC 值, 观察仿真过程中 PC 的变化
initial begin
    $monitor("Time: %t | currentPC: %h | nextpc: %h | PCSrc: %b |
PCWre: %b | immediate: %h | jumppc: %h | Reset: %b",
            $time, currentPC, nextpc, PCSrc, PCWre, immediate, jumppc,
Reset);
end
endmodule

```

仿真波形:



6.RegFile

寄存器组负责从寄存器的读写操作，该模块接受 9 个输入：

- CLK：时钟信号，当时钟下降沿时才能进行写操作
- RegDst：RegDst 为真时，处理 R 型指令，rd 为目标操作数寄存器，为假时处理 I 型指令，rt 为目标操作数寄存器
- RegWre：为 1 时才允许进行写操作
- DBDataSrc：选择写的数据是来自 ALU 还是数据存储器
- rs：rs 寄存器地址
- rt：rt 寄存器地址
- rd：rd 寄存器地址
- dataFromALU：来自 ALU 输出的数据
- dataFromM：来自数据存储器的数据

该模块输出为：

- busA：输出的第一个数
- busB：输出的第二个数
- writeReg：要进行写的数，放在输出端口为了在 7 段数码管上显示

模块代码：

```
module RegFile(//控制信号: RegWre, RegDst, DBDataSrc
    input CLK, RegDst, RegWre, DBDataSrc,
    input [4:0] rs, rt, rd,
    input [31:0] dataFromALU, dataFromM,

    output [31:0] busA, busB,
    output [31:0] writeData
);
    wire [4:0] writeReg; //写入寄存器的编号

    assign writeReg = RegDst ? rd : rt;
    assign writeData = DBDataSrc ? dataFromM : dataFromALU;

    reg [31:0] register[0:31];
    integer i;
    initial begin
        for(i = 0; i < 32; i=i+1 )
            register[i] <= 32'b0;
    end
    //判断指令类型

    assign busA = register[rs];
    assign busB = register[rt];

    always@(negedge CLK) begin
        if(RegWre && writeReg) begin
            register[writeReg] <= writeData;
        end
    end
endmodule
```

仿真代码：

```
module RegFile_tb;
    // Inputs
    reg CLK;
    reg RegDst;
    reg RegWre;
    reg DBDataSrc;
    reg [4:0] rs, rt, rd;
    reg [31:0] dataFromALU, dataFromM;
```

```

// Outputs
wire [31:0] busA, busB, writeData;

// Instantiate the RegFile module
RegFile uut (
    .CLK(CLK),
    .RegDst(RegDst),
    .RegWre(RegWre),
    .DBDataSrc(DBDataSrc),
    .rs(rs),
    .rt(rt),
    .rd(rd),
    .dataFromALU(dataFromALU),
    .dataFromM(dataFromM),
    .busA(busA),
    .busB(busB),
    .writeData(writeData)
);

// Clock generation
always begin
    #5 CLK = ~CLK; // Period is 10 time units
end

// Stimulus block
initial begin
    // Initialize Inputs
    CLK = 0;
    RegDst = 0;
    RegWre = 0;
    DBDataSrc = 0;
    rs = 5'd0; // Register 0
    rt = 5'd1; // Register 1
    rd = 5'd2; // Register 2
    dataFromALU = 32'h12345678;
    dataFromM = 32'h87654321;

    #10;

    RegWre = 1;
    RegDst = 1; // Write to rd
    DBDataSrc = 0; // Use ALU result
    rd = 5'd3; // Write to register 3

```

```

#10; // Wait for one clock cycle

RegDst = 0; // Write to rt
DBDataSrc = 1; // Use Data memory result
rt = 5'd4; // Write to register 4
#10;

rs = 5'd3; // Read from register 3
rt = 5'd4; // Read from register 4
#10;

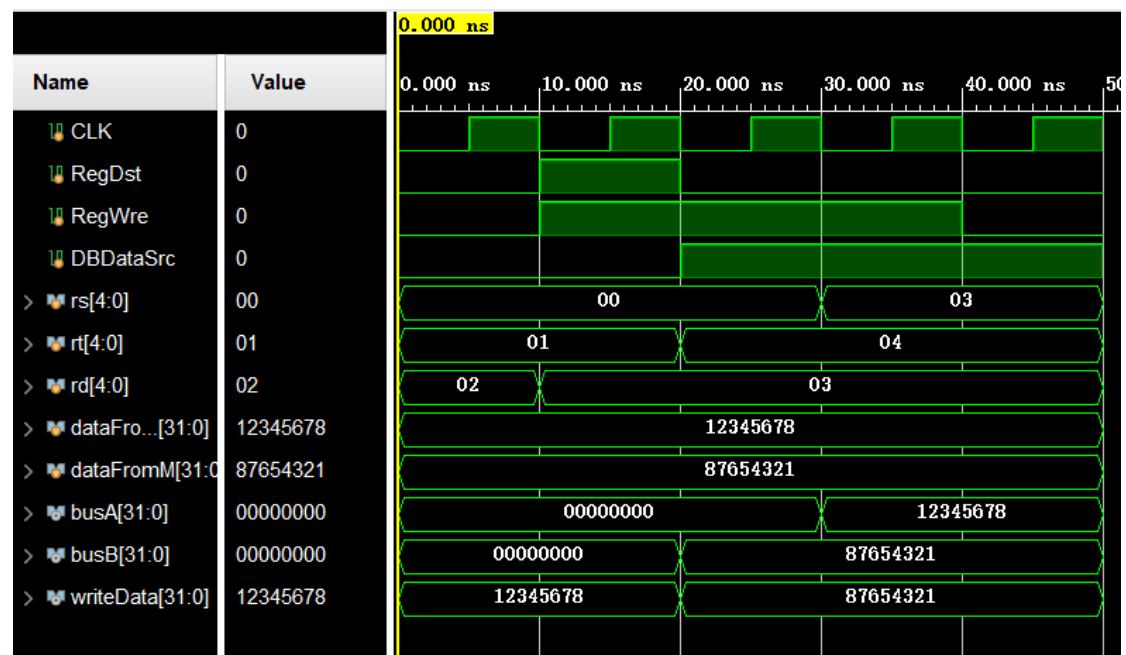
RegWre = 0; // No writes allowed
#10;

$finish;
end

// Monitor signals
initial begin
    $monitor("Time: %0t | RegDst: %b | RegWre: %b | DBDataSrc: %b |
rs: %d | rt: %d | rd: %d | busA: %h | busB: %h | writeData: %h",
            $time, RegDst, RegWre, DBDataSrc, rs, rt, rd, busA, busB,
writeData);
end
endmodule

```

仿真波形:



7. SignZeroExt

符号拓展单元对 16 位立即数按符号拓展为 32 位的数据，用于跳转指令和 I 型指令。

输入：

- immediate : 所要拓展的 16 位立即数
- ExtSel : 为 1 高位补符号位，为 0 高位补 0

输出：

- ext: 32 位扩展后的立即数

模块代码：

```
module SignZeroExt(//控制信号: Extsel
    input  [15:0] immediate,
    input  Extsel, //1sign00
    output [31:0] ext
);

    assign ext[15:0] = immediate[15:0];
    assign ext[31:16] = Extsel ? {16{immediate[15]}} : 16'b0;

endmodule
```

仿真代码：

```
module SignZeroExt_tb;

    // 定义信号
    reg [15:0] immediate; // 16 位立即数
    reg Extsel;           // 控制信号
    wire [31:0] ext;       // 扩展后的信号

    // 实例化 SignZeroExt 模块
    SignZeroExt uut (
        .immediate(immediate),
        .Extsel(Extsel),
    );

endmodule
```

```

        .ext(ext)
    );

// 仿真过程
initial begin
    // 初始化信号
    immediate = 16'b0000000000000001; // 一个正数
    Extsel = 0; // 零扩展
    #10; // 等待 10 个时间单位
    $display("immediate = %b, Extsel = %b, ext = %b", immediate,
Extsel, ext);

    immediate = 16'b1111111111111111; // 一个负数
    Extsel = 0; // 零扩展
    #10; // 等待 10 个时间单位
    $display("immediate = %b, Extsel = %b, ext = %b", immediate,
Extsel, ext);

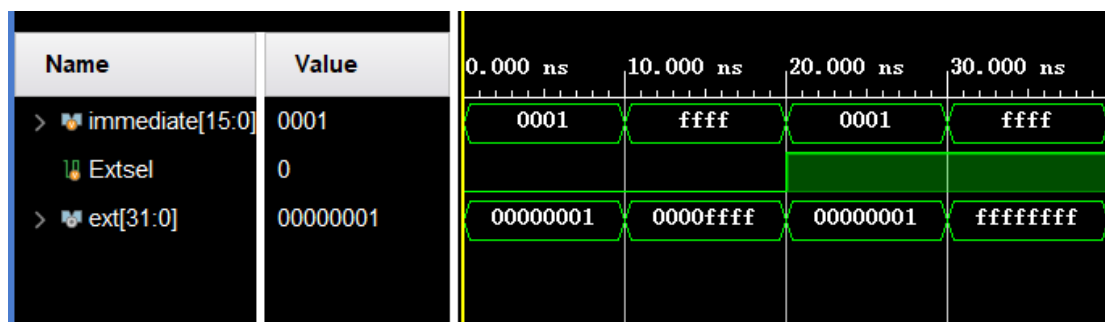
    immediate = 16'b0000000000000001; // 一个正数
    Extsel = 1; // 符号扩展
    #10; // 等待 10 个时间单位
    $display("immediate = %b, Extsel = %b, ext = %b", immediate,
Extsel, ext);

    immediate = 16'b1111111111111111; // 一个负数
    Extsel = 1; // 符号扩展
    #10; // 等待 10 个时间单位
    $display("immediate = %b, Extsel = %b, ext = %b", immediate,
Extsel, ext);

    $finish; // 仿真结束
end
endmodule

```

仿真波形：



8. CPU

设计完各个模块之后，接下来的工作是将各个模块的接口连接起来，组合成一个完整的 CPU 模块。

输入：

- CLK
- Reset: 为 1 时 pc 清零

输出：

- rs, rt, Opcode, ra, rb, currentPC, nextpc, result
- DBData: DataMemory 输出的数据

模块代码：

```
module CPU1(  
    input CLK,  
    input Reset,  
    output[4:0]rt,rs,  
    output wire [5:0]Opcode,  
  
    output wire [31:0] ra,rb, currentPC, nextpc, result,  
    DBData//,instruction  
);  
    wire [5:0]funct;  
    wire [2:0] ALUOp;  
    wire [3:0] aluop2;  
    wire [31:0]immediateExt;  
    wire [31:0]dataout;//是 datamem 出来的内容  
    wire [15:0]immediate;  
    wire [4:0]rd;  
    wire[4:0]shamt;  
    wire [31:0]jumppc;  
    wire zero,sign, PCWre,ALUSrcA,ALUSrcB,DBDataSrc,RegWre;  
    wire InsMemRW,mRD,mWR,Exsel,RegDst;  
    wire [1:0] PCSrc;  
    wire [3:0]PC4;
```


//各模块实例化

```
    pc pc_inst (
        .PCSrc(PCSrc),          // PCSrc 信号决定 PC 的选择
        .CLK(CLK),              // 时钟信号
        .PCWre(PCWre),          // 控制 PC 写使能
        .immediate(immediate), // branch 的偏移量
        .jumppc(jumppc),        // 跳转地址
        .Reset(Reset),          // 复位信号
        .currentPC(currentPC), // 当前 PC
        .nextpc(nextpc),        // 下一 PC
        .pc4(PC4)                // 当前 PC 的低 4 位
    );

    ctr ctr_inst (
        .opCode(Opcode),
        .zero(zero),
        .sign(sign),
        .PCWre(PCWre),
        .ALUSrcB(ALUSrcB),
        .DBDataSrc(DBDataSrc),
        .RegWre(RegWre),
        .InsMemRW(InsMemRW),
        .mRD(mRD),
        .mWR(mWR),
        .RegDst(RegDst),
        .ExtSel(Exsel), // ExtSel 连接
        .PCSrc(PCSrc),
        .aluop(aluop2)
    );

    aluctr alu_control_inst (
        .aluop(aluop2),
        .funct(funct),
        .ALUOp(ALUOp),
        .ALUSrcA(ALUSrcA)
    );

    ALU alu_instance (
        .ra(ra),          // Connect ra signal from CPU1
        .rb(rb),          // Connect rb signal from CPU1
        .shamt(shamt),    // Connect shamt signal from CPU1
        .immediateExt(immediateExt), // Connect immediateExt from CPU1
        .ALUOp(ALUOp),    // ALU operation code (3-bit control signal)
        .ALUSrcA(ALUSrcA), // ALU source A selector
    );
```

```

        .ALUSrcB(ALUSrcB), // ALU source B selector
        .result(result),    // ALU result
        .zero(zero) ,       // Zero flag output from ALU
        .sign(sign)
    );

    DataMemory u_dataMemory(
        .CLK(CLK),
        .mWR(mWR),
        .mRD(mRD),
        .raAddIm(result), // 可能需要确认是否为`result`作为地址输入
        .datain(rb), // 这里我们假设写入数据是`rb`
        .dataout(dataout)
    );

    InstructionMemory u_instMem(
        .pc4(PC4),
        .funct(funct),
        .InsMemRW(InsMemRW),
        .currentPC(currentPC),
        .op(Opcode),
        .rs(rs),
        .rt(rt),
        .rd(rd),
        .shamt(shamt),
        .immediate(immediate),
        .jumppc(jumppc)
    );

    RegFile u_RegFile (
        .CLK(CLK), // 连接时钟信号
        .RegDst(RegDst), // 连接 RegDst 控制信号
        .RegWre(RegWre), // 连接 RegWre 控制信号
        .DBDataSrc(DBDataSrc), // 连接 DBDataSrc 控制信号
        .rs(rs), // 连接源寄存器 rs 地址
        .rt(rt), // 连接源寄存器 rt 地址
        .rd(rd), // 连接目标寄存器 rd 地址
        .dataFromALU(result), // 连接 ALU 输出数据
        .dataFromM(dataout), // 连接内存输出数据
        .busA(ra), // 连接读出的 busA 数据
        .busB(rb), // 连接读出的 busB 数据
        .writeData(DBData) // 连接写入寄存器的数据
    );

    SignZeroExt u_signZeroExt(

```

```

        .immediate(immediate),
        .Extsel(Exsel),
        .ext(immediateExt)
    );
Endmodule

```

仿真代码:

```

module CPU1_sim;

    // 定义输入信号
    reg CLK;
    reg Reset;

    // 定义输出信号
    // wire [4:0] rt, rs;
    wire [5:0] Opcode;
    wire [31:0] ra, rb, currentPC, result;//, DBData,nextpc,,instruction;

    // 实例化 SingleCycleCPU 模块
    CPU1 uut (
        .CLK(CLK),
        .Reset(Reset),
        .Opcode(Opcode),
        .ra(ra),
        .rb(rb),
        .currentPC(currentPC),
        .result(result)
    );

    initial begin
        //initial inputs
        CLK = 0;
        Reset = 0;//刚开始设置 PC 为 0

        #5;
        CLK = 1;

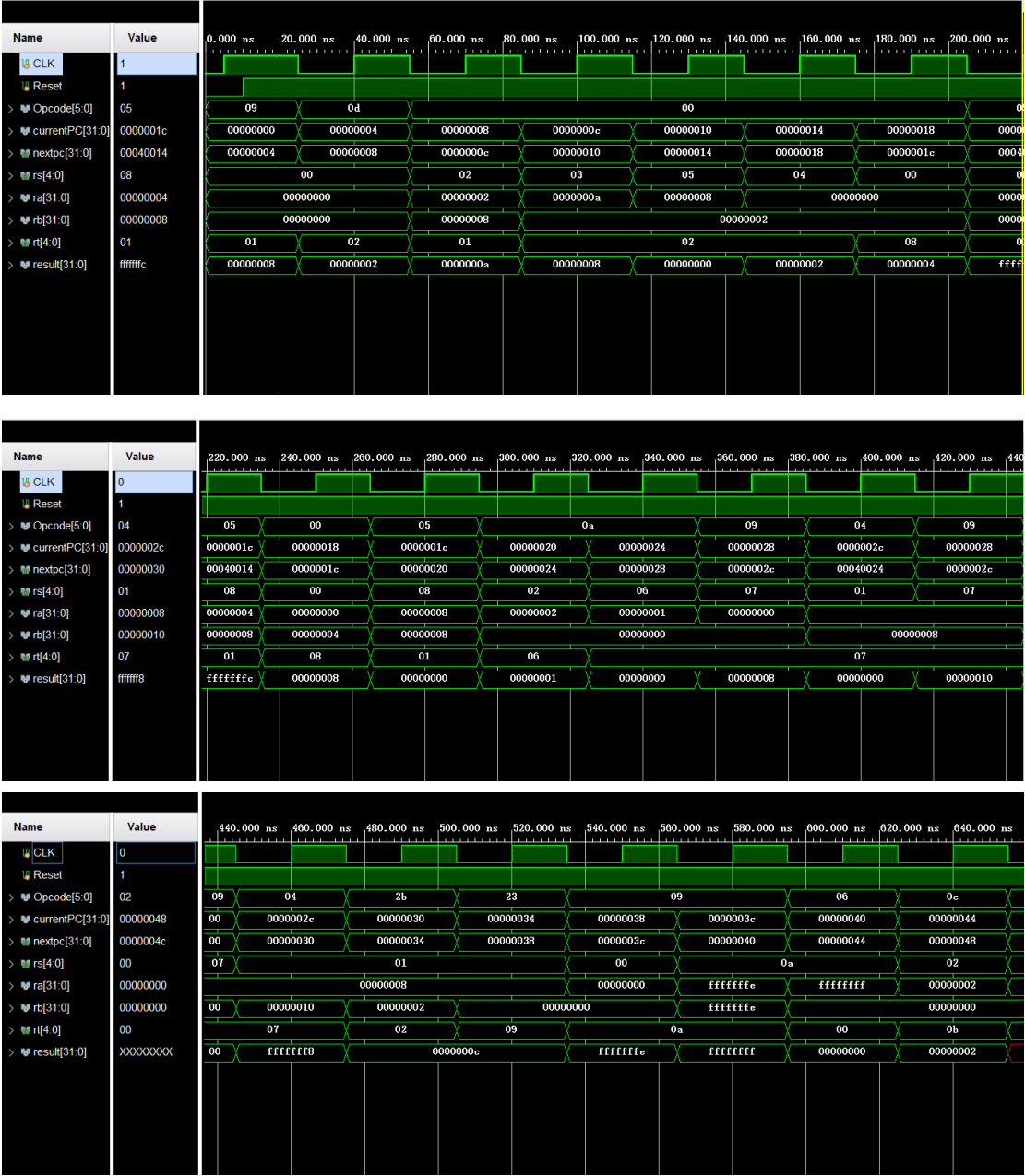
        #5;
        Reset = 1;

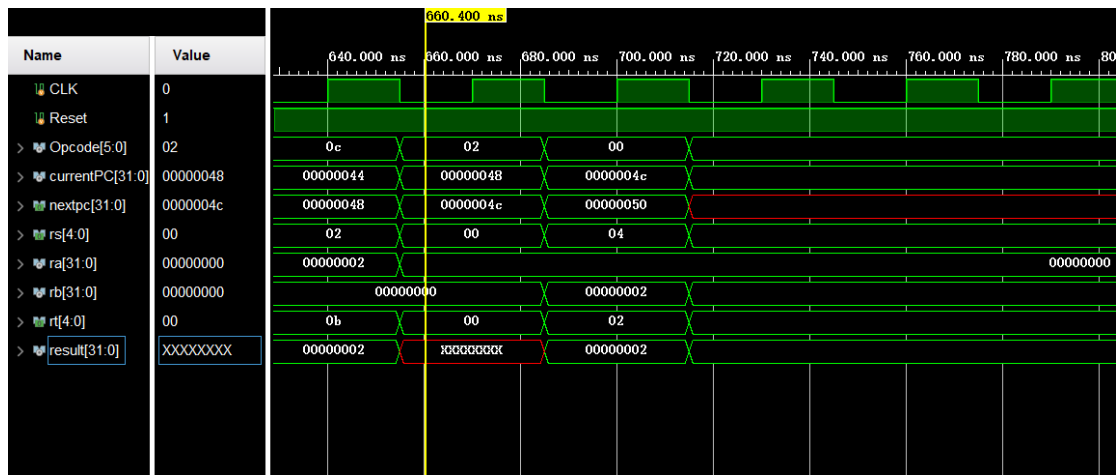
        //产生时钟信号
        forever #15 begin
            CLK = !CLK;

```

```
end
end
endmodule
```

仿真波形：





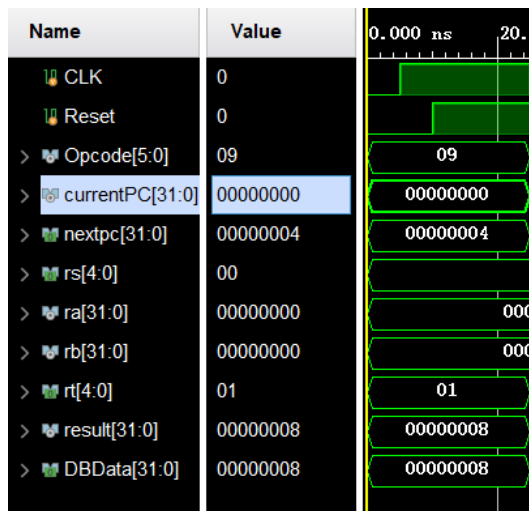
其中要执行的指令存储在 Instructions.mem 文件中，指令内容和对应的机器码为：

addiu \$1,\$0,8	001001	00000	00001	0000000000001000
ori \$2,\$0,2	001101	00000	00010	0000000000000010
add \$3,\$2,\$1	000000	00010	00001	0001100000100000
sub \$5,\$3,\$2	000000	00011	00010	0010100000100010
and \$4,\$5,\$2	000000	00101	00010	0010000000100100
or \$8,\$4,\$2	000000	00100	00010	0100000000100101
sll \$8,\$8,1	000000	00000	01000	01000000010 0000
bne \$8,\$1,-2 (≠,转 18)	000101	01000	00001	1111111111111110
slti \$6,\$2,4	001010	00010	00110	0000000000000100
slti \$7,\$6,0	001010	00110	00111	0000000000000000
addiu \$7,\$7,8	001001	00111	00111	0000000000001000
beq \$7,\$1,-2 (=,转 28)	000100	00001	00111	1111 1111 1111 1110
sw \$2,4(\$1)	101011	00001	00010	0000000000000100
lw \$9,4(\$1)	100011	00001	01001	0000000000000100
addiu \$10,\$0,-2	001001	00000	01010	1111 1111 1111 1110
addiu \$10,\$10,1	001001	01010	01010	0000000000000001
blez \$10,-2(≤0,转 3C)	000110	01010	00000	1111 1111 1111 1110
andi \$11,\$2,2	001100	00010	01011	0000000000000010
j 0x0000004C	000010	00000	00000	0000000000010011
or \$8,\$4,\$2	000000	00100	00010	0100000000100101
halt	111111	00000	00000	0000000000000000

addiu 指令

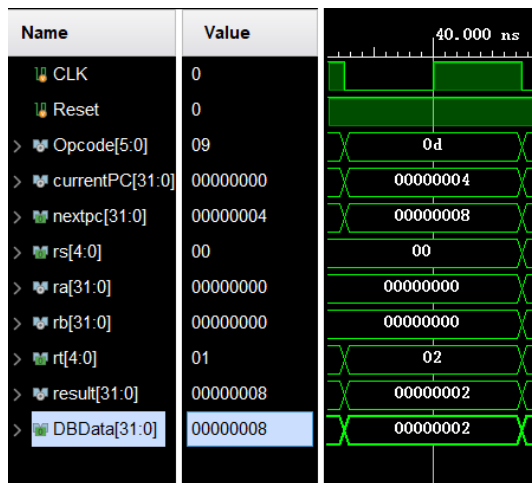
addiu \$1,\$0,8

001001	00000	00001	0000000000001000
--------	-------	-------	------------------



Ori 指令

ori \$2,\$0,2	001101	00000	00010	00000000000000010
---------------	--------	-------	-------	-------------------



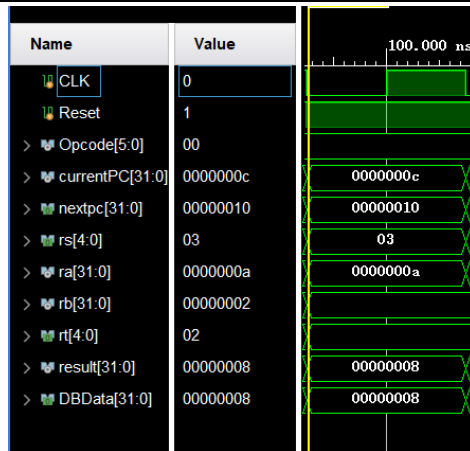
Add 指令

add \$3,\$2,\$1	000000	00010	00001	0001100000100000
-----------------	--------	-------	-------	------------------



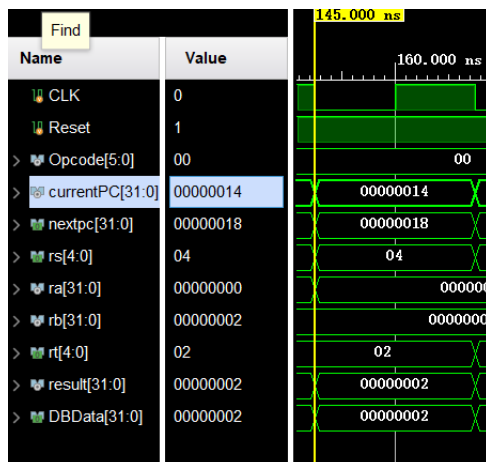
Sub 指令

sub \$5,\$3,\$2	000000	00011	00010	0010100000100010
-----------------	--------	-------	-------	------------------



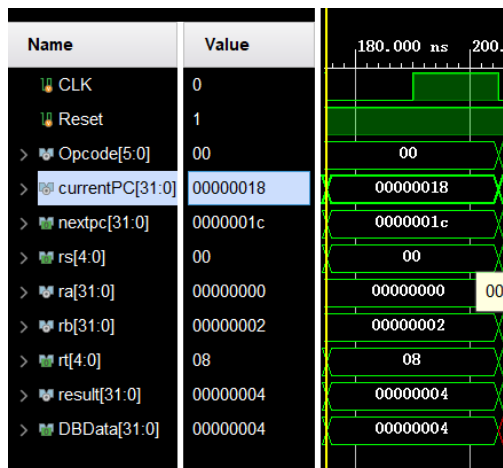
Or 指令

or \$8,\$4,\$2	000000	00100	00010	0100000000100101
----------------	--------	-------	-------	------------------



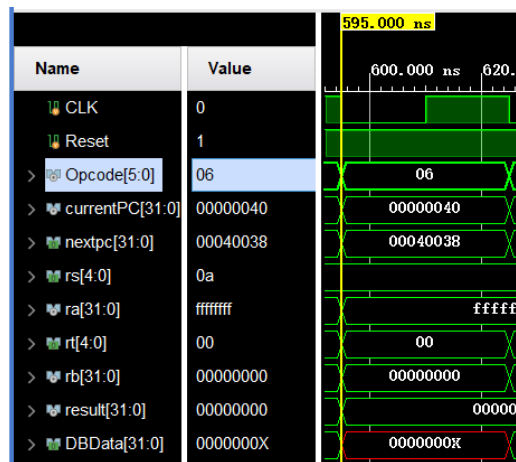
Sll 指令

sll \$8,\$8,1	000000	00000	01000	01000000010 0000
---------------	--------	-------	-------	------------------



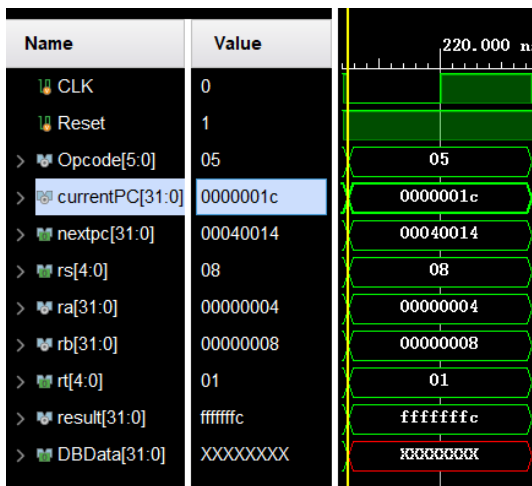
Blez 指令

blez \$10,-2(≤0,转 3C)	000110	01010	00000	1111 1 111 1111 1110
-----------------------	--------	-------	-------	----------------------



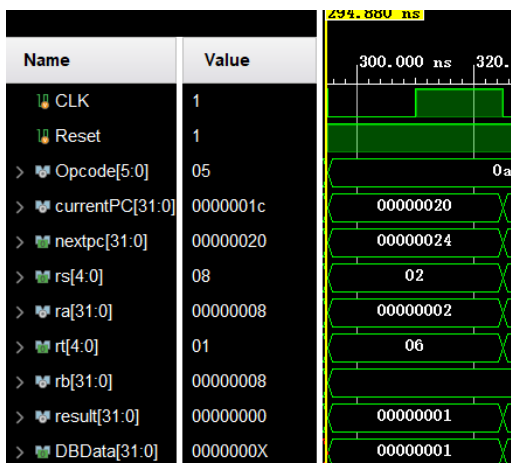
Bne 指令

bne \$8,\$1,-2 (≠,转 18)	000101	01000	00001	1111111111111110
-------------------------	--------	-------	-------	------------------



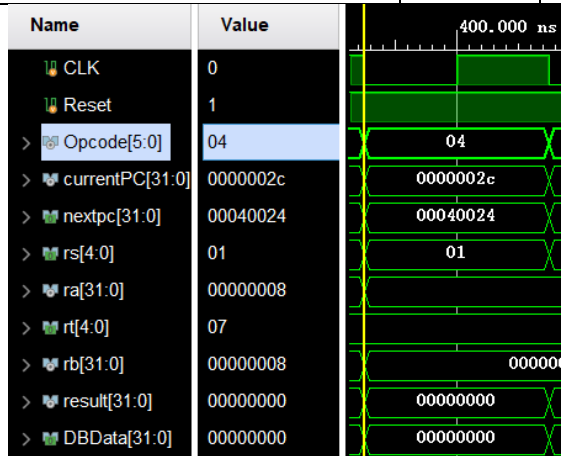
Slti 指令

slti \$6,\$2,4	001010	00010	00110	00000 00000000100
----------------	--------	-------	-------	-------------------



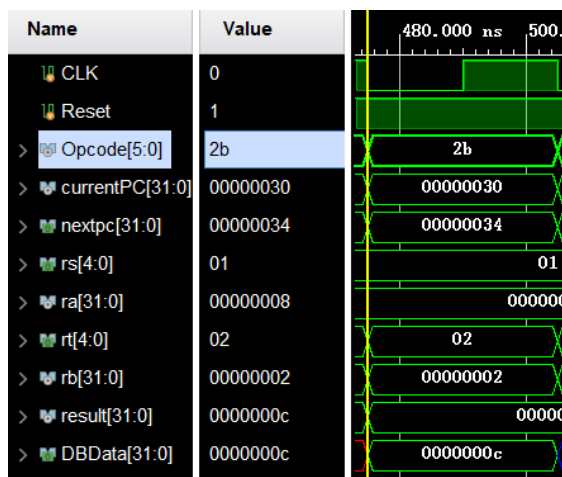
Beq 指令

beq \$7,\$1,-2 (=,转 28)	000100	00001	00111	1111 1 111 1111 1110
-------------------------	--------	-------	-------	----------------------



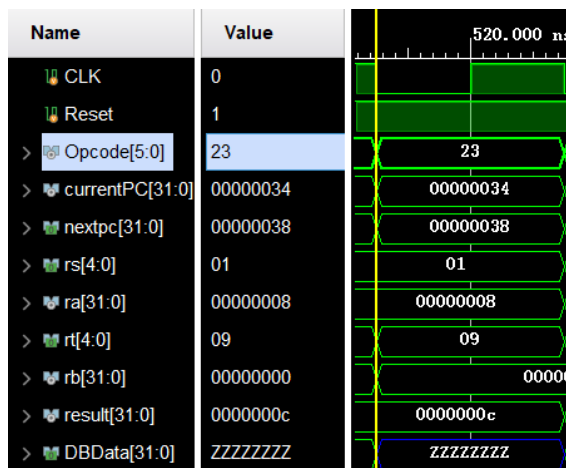
sw 指令

sw \$2,4(\$1)	101011	00001	00010	00000 00000000100
---------------	--------	-------	-------	-------------------



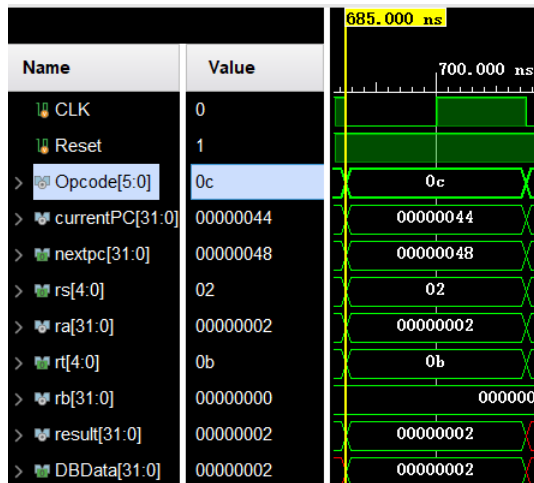
Lw 指令

lw \$9,4(\$1)	100011	00001	01001	00000 00000000100
---------------	--------	-------	-------	-------------------



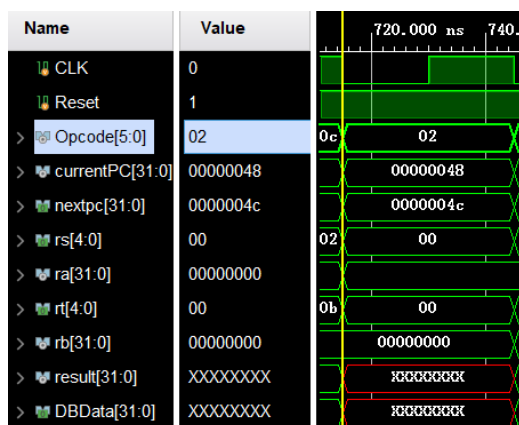
Andi 指令

andi \$11,\$2,2	001100	00010	01011	00000 00000000010
-----------------	--------	-------	-------	-------------------



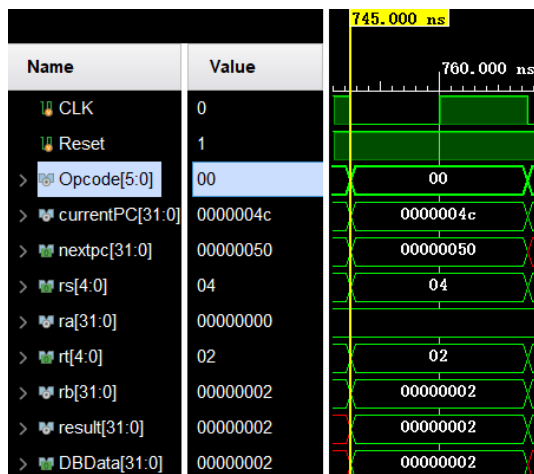
Jump 指令

j 0x0000004C	000010	00000	00000	00000 00000010011
--------------	--------	-------	-------	-------------------



Halt 指令

halt	111111	00000	00000	00000 00000000000
------	--------	-------	-------	-------------------



10. 进行 Basys 烧写开发板

设计思想：

- 实现 CPU 在板上运行需要两个时钟信号，CPU 工作时钟和 Basys3 板系统时钟。CPU 工作时钟即为按键，是 CPU 正常工作时钟信号，按键必须进行消抖处理；Basys3 板系统时钟即为板提供的正常工作时钟信号，即为 100MHZ。Basys3 板系统时钟信号引脚 对应管脚 W5。
- 每个按键周期，4 个数码管都必须刷新一次。数码管位控信号 AN3-AN0 是 1110、1101、1011、0111，为 0 时点亮该数码管，当然，还应该为数码管各位 “1gfedcba” 引脚输出信号，最高位为 “1”。比如，“当前 PC 值” 低 8 位中的高 4 位和低 4 位，必须经下页转换后送给数码管各引脚。

综上，显示模块大概分为 4 个部分：

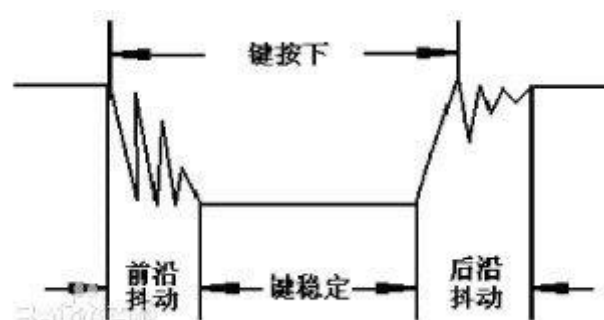
- 对 Basys3 板系统时钟信号进行分频，分频的目的用于计数器
- 生成计数器，计数器用于产生 4 个数。这 4 数用于控制 4 个数码管
- 根据计数器产生的数生成数码管相应的位控信号(输出)和接收 CPU 来的相应数据
- 将从 CPU 接收到的相应数据转换为数码管显示信号，再送往数码管显示(输出)

分频模块

时钟分频的原理是定义一个计数器 cnt，每当原始时钟（开发板的时钟 BasysCLK）到来时加一。当计数器满 $2^N - 1$ 后，再将分频时钟 CLK_SLow 翻转一次，这样就将 CLK 进行了 2^N 分频

```
module CLK_slow(  
    input CLK_100mhz, //100mhz 的原始时钟信号  
  
    output reg CLK_slow //输出的分频时钟信号  
);  
  
reg [31:0] count = 0; //计数器  
reg [31:0] N = 50000; //分频  $2^{19}$ ，得到一个周期大约 5ms 的信号  
  
initial CLK_slow = 0;  
  
always @ (posedge CLK_100mhz) begin  
    if(count >= N) begin  
        count <= 0;  
        CLK_slow <= ~CLK_slow;  
    end  
    else  
        count <= count + 1;  
  
end  
endmodule
```

消抖模块



为什么要消抖？见下图：

我们可以看到，在按键按下的那一刻，存在一段时间的抖动，同时在释放按键的一段时间里也是存在抖动的，这就可能导致状态在识

别的时候可能检测为多次的按键，因为运行过程中普通的检测一次状态 key 为 1 就执行一次按键操作。所以我们在使用按键时往往需要消抖。

输入消抖模块同样使用了 N 位一个计数器 cnt，当时钟信号到来时，如果按键处于按下状态就将计数器+1，而如果现按键未按下，则清空计数器。将模块输出的防抖按键信号连接到 cnt 第 N 位，这样就保证持续按下 2^N 个时钟周期的时间才会输出按键信号，从而屏蔽了短暂的抖动信号。

```
module Keyboard_CLK(  
    input Button,  
    input BasysCLK,  
  
    output CPUCLK  
);  
  
reg button_previous_state;  
reg button_current_state;  
  
wire button_edge;  
  
always @ (posedge BasysCLK) begin  
    button_current_state <= Button;  
    button_previous_state <= button_current_state;  
end  
  
assign button_edge = button_previous_state & (~button_current_state);  
  
reg [20:0] counter;  
  
always @ (posedge BasysCLK) begin  
    if(button_edge)  
        counter <= 21'h0;  
    else  
        counter <= counter + 1;  
end
```

```

end

reg delayed_button_previous_state;
reg delayed_button_current_state;

//21'h1E8480
always @ (posedge BasysCLK) begin
    if(counter == 21'h1E8480)
        delayed_button_current_state <= button_current_state;
        delayed_button_previous_state <= delayed_button_current_state;
end

assign CPUCLK = delayed_button_previous_state &
(~delayed_button_current_state);

endmodule

```

显示模块

原理是，当输入一个 4 位的控制信号 `display_data` 时，输出一个 8 位的信号，将这个信号输入给 7 段数码管的引脚，就可以让数码管输出对应的数字。

```

module Display_7SegLED(
    input [3:0] display_data,
    output reg [7:0] dispcode
);

always @ (display_data) begin
    case(display_data)
        4'b0000 : dispcode = 8'b1100_0000; //0;'0'-亮灯, '1'-熄灯
        4'b0001 : dispcode = 8'b1111_1001; //1
        4'b0010 : dispcode = 8'b1010_0100; //2
        4'b0011 : dispcode = 8'b1011_0000; //3
        4'b0100 : dispcode = 8'b1001_1001; //4
        4'b0101 : dispcode = 8'b1001_0010; //5
        4'b0110 : dispcode = 8'b1000_0010; //6
        4'b0111 : dispcode = 8'b1101_1000; //7
        4'b1000 : dispcode = 8'b1000_0000; //8
        4'b1001 : dispcode = 8'b1001_0000; //9
        4'b1010 : dispcode = 8'b1000_1000; //A
    endcase
end

```

```

        4'b1011 : dispcode = 8'b1000_0011; //b
        4'b1100 : dispcode = 8'b1100_0110; //C
        4'b1101 : dispcode = 8'b1010_0001; //d
        4'b1110 : dispcode = 8'b1000_0110; //E
        4'b1111 : dispcode = 8'b1000_1110; //F
        default : dispcode = 8'b0000_0000; //不亮
    endcase
end

endmodule

```

选择模块

接受 4 个要显示的数据和一个决定显示哪个数据的控制信号

SelectCode, 然后输出对应的 display_data 给显示模块

```

module Select(
    input [15:0] In1, In2, In3, In4,
    input [1:0] SelectCode,

    output reg [15:0] DataOut
);

always @ (*) begin
    case (SelectCode)
        2'b00 : DataOut = In1;
        2'b01 : DataOut = In2;
        2'b10 : DataOut = In3;
        2'b11 : DataOut = In4;
    endcase
end

endmodule

```

转换模块

对输入的时钟信号和数据进行转换并输出

```

module Transfer(
    input CLK,
    input [15:0] In,

```

```

        output reg [3:0] Out,
        output reg [3:0] Bit
    );

integer i;
initial begin
    i = 0;
end

always @ (negedge CLK) begin
    case(i)
        0 : begin
            Out = In[15:12];
            Bit = 4'b1110;
        end
        1 : begin
            Out = In[11:8];
            Bit = 4'b1101;
        end
        2 : begin
            Out = In[7:4];
            Bit = 4'b1011;
        end
        3 : begin
            Out = In[3:0];
            Bit = 4'b0111;
        end
    endcase
    i = (i == 3) ? 0 : i + 1;
end
endmodule

```

Basys3 模块：连接所有模块

Basys3 模块连接以上 5 个与显示有关模块和 CPU 模块，最终烧写进开发板里面。

```

module Basys3(
    input CLKButton,
    input BasysCLK,
    input RST_Button,

```



```

        input [1:0] SW_in,

        output [7:0] SegOut,
        output [3:0] Bits
    );

    //CPU
    wire [4:0] RsAddr, RtAddr;
    wire [31:0] RsData, RtData;
    wire [31:0] ALUResult;
    wire [31:0] DBData;
    wire [31:0] curPC, nextPC;

    wire CPUCLK;

    CPU1 CPU1(
        .CLK(CPUCLK),
        .Reset(RST_Button),
        .rs(RsAddr),
        .rt(RtAddr),
        .Opcode(op),
        .ra(RsData),
        .rb(RtData),
        .currentPC(curPC),
        .nextpc(nextPC),
        .result(ALUResult),
        .DBData(DBData)
    );

    //CLK_slow

    wire Div_CLK;
    CLK_slow clk_slow(
        .CLK_100mhz(BasysCLK),
        .CLK_slow(Div_CLK)
    );

    //Display_7Seg
    wire [3:0] SegIn;

    Display_7SegLED display_led(
        .display_data(SegIn),
        .dispcode(SegOut)
    );

```

```

//Display_select

wire [15:0] display_data;
Select select(
    .In1({curPC[7:0], nextPC[7:0]}),
    .In2({3'b000, RsAddr[4:0], RsData[7:0]}),
    .In3({3'b000, RtAddr[4:0], RtData[7:0]}),
    .In4({ALUResult[7:0], DBData[7:0]}),

    .SelectCode(SW_in),
    .DataOut(display_data)
);

//Display_transfer
Transfer transfer(
    .CLK(Div_CLK),
    .In(display_data),

    .Out(SegIn),
    .Bit(Bits)
);

//keyboard
Keyboard_CLK keyboard(
    .Button(CLKButton),
    .BasysCLK(BasysCLK),
    .CPUCLK(CPUCLK)
);
endmodule

```

同时进行了测试，测试中认为产生按键时钟，测试代码为：

```

module basys_sim(

);
reg CLK, RST;
reg CLKButton;
reg [1:0] SW;

Basys3 test(
    .BasysCLK(CLK),
    .CLKButton(CLKButton),
    .SW_in(SW),

```

```

        .RST_Button(RST),

        .SegOut(),
        .Bits()
    );
    integer i;

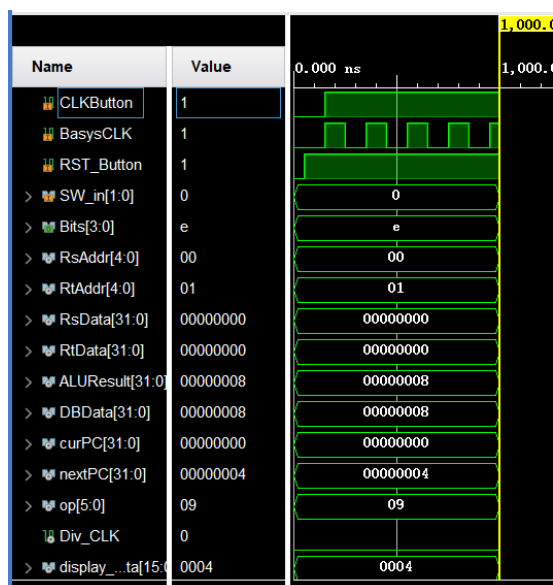
    initial begin
        //recode
        $dumpfile("Basys3.vcd");
        $dumpvars(0, basys_sim);

        CLK = 0;
        RST = 0;
        CLKButton = 0;
        SW = 2'b00;

        #50;
        RST = 1;
        for (i = 0; i < 1000000; i = i + 1) begin
            #100;
            CLK = ~CLK;
//            #10;
            if (i % 15 == 0) CLKButton = ~CLKButton;
        end
    end
endmodule

```

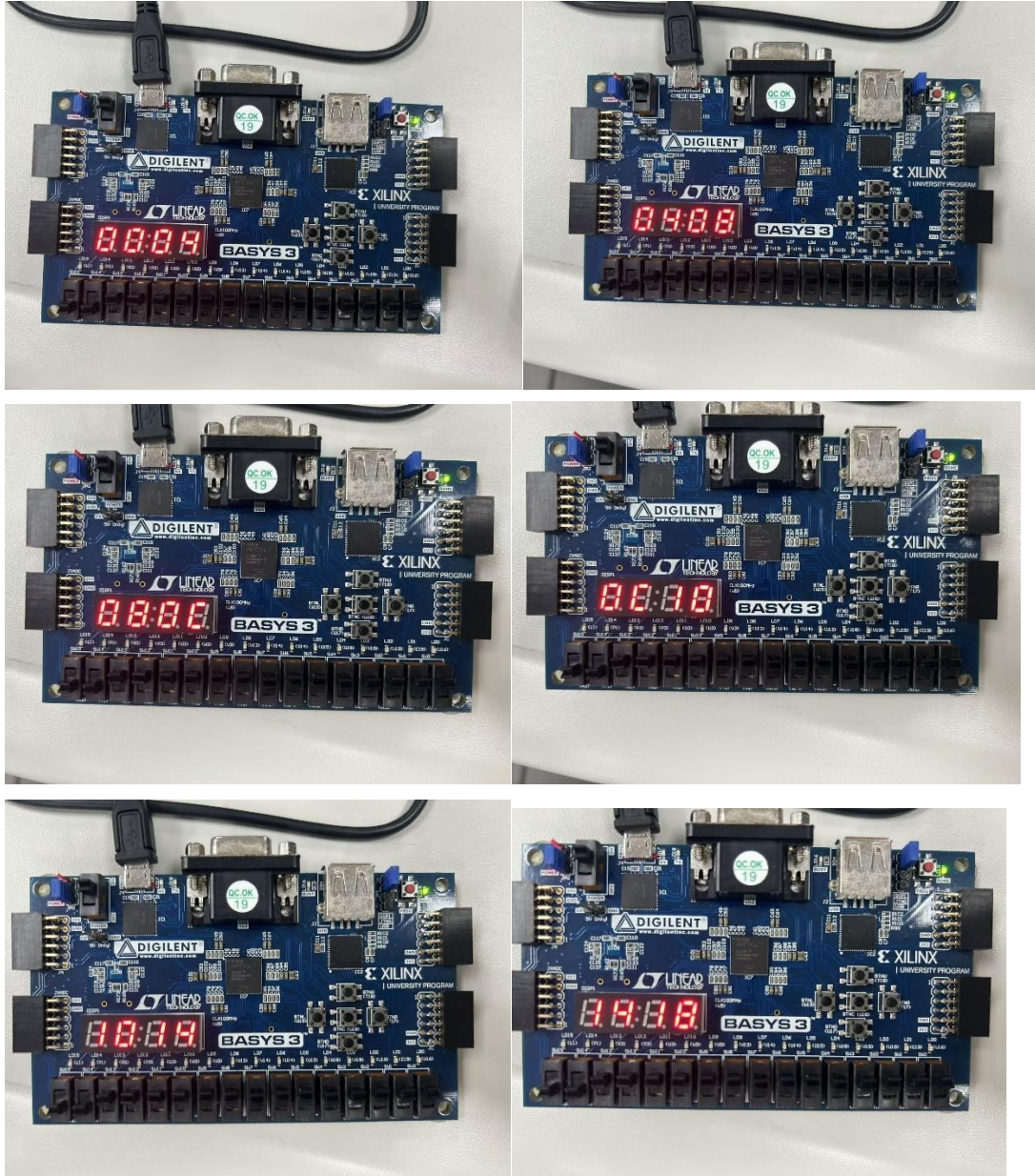
产生的波形为：



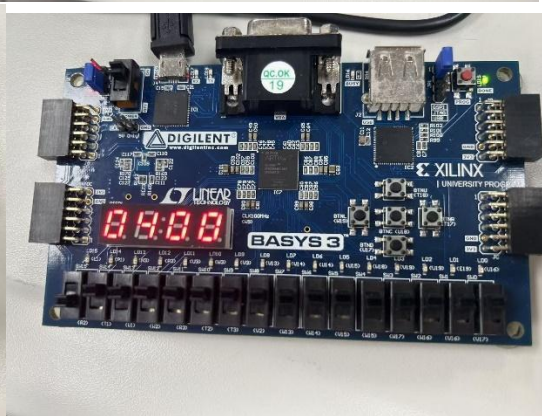
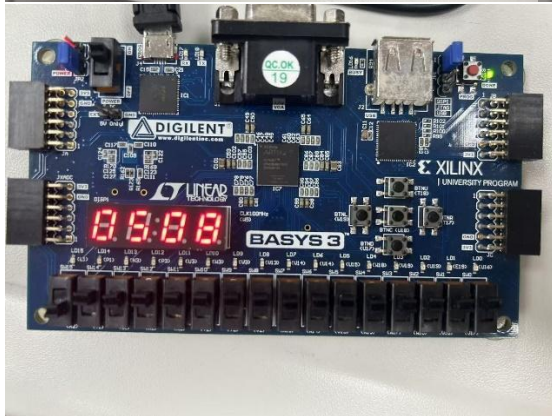
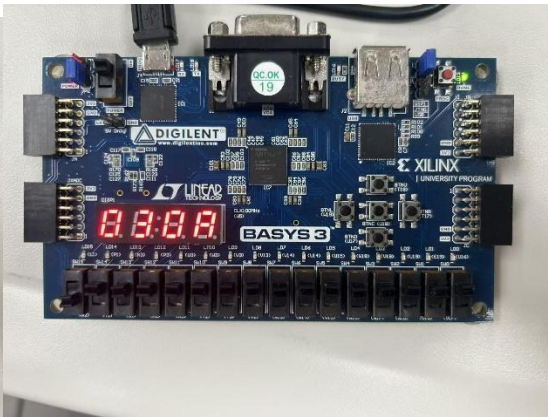
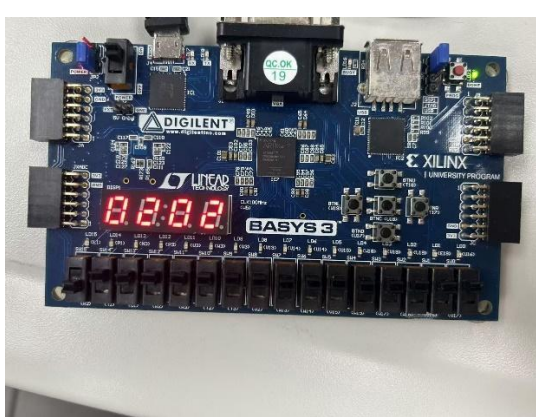
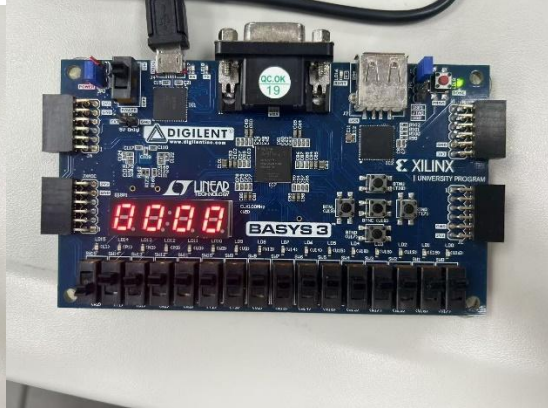
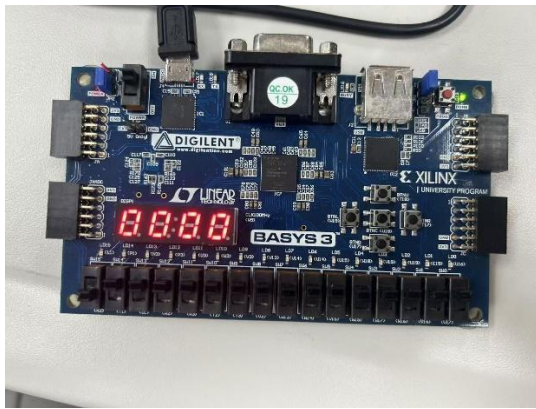
11. 烧板

仅列举前 6 条指令：

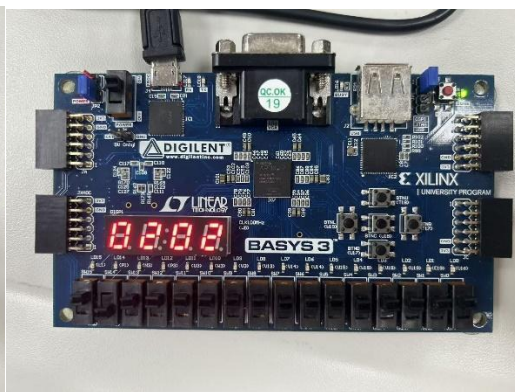
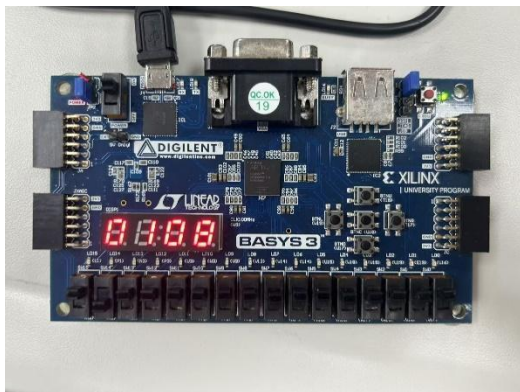
- PC 和 nextPC

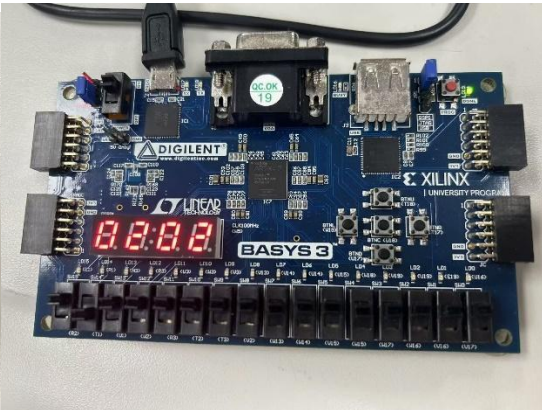
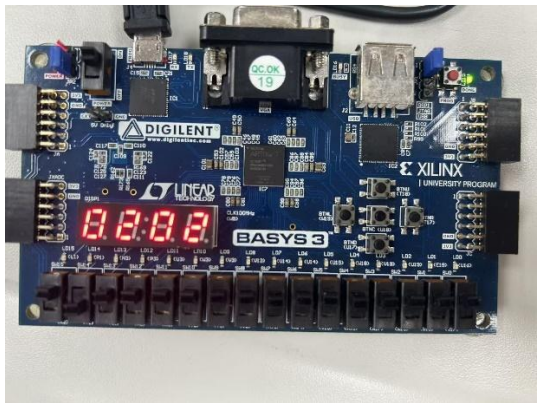
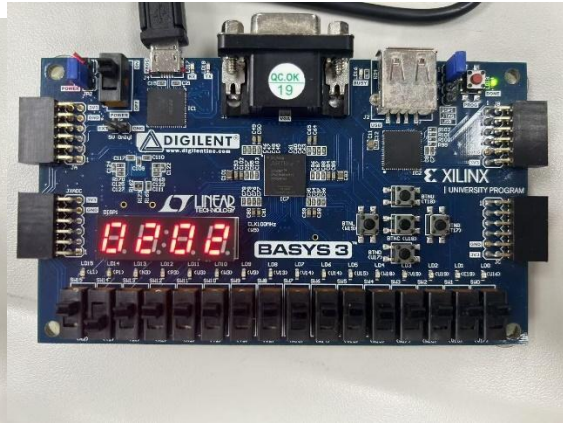
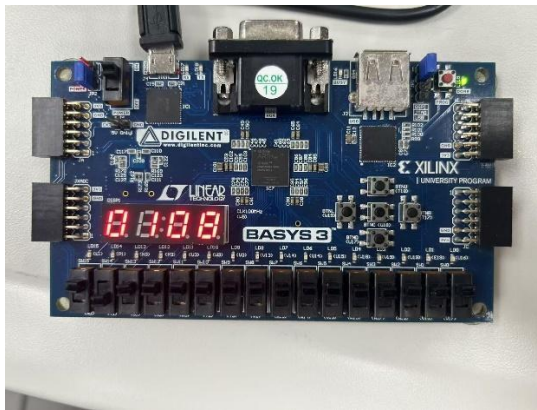


- Rs 的编号和内容

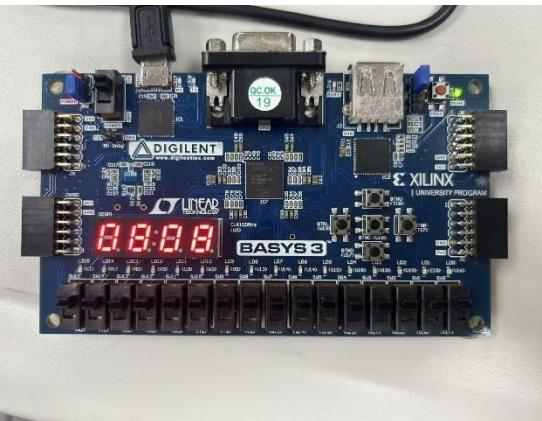
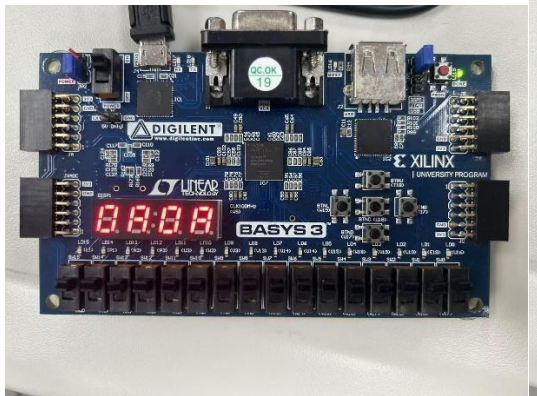
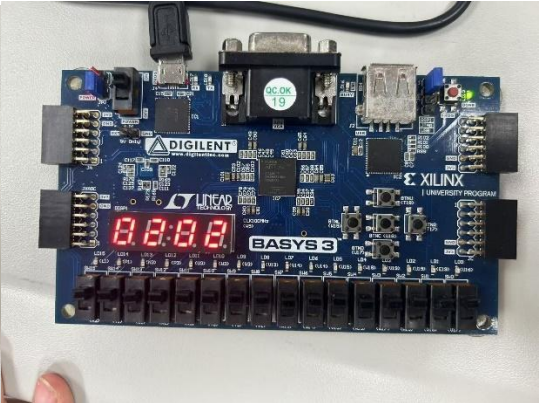
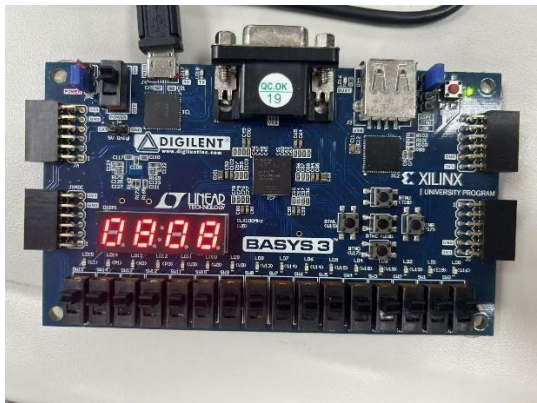


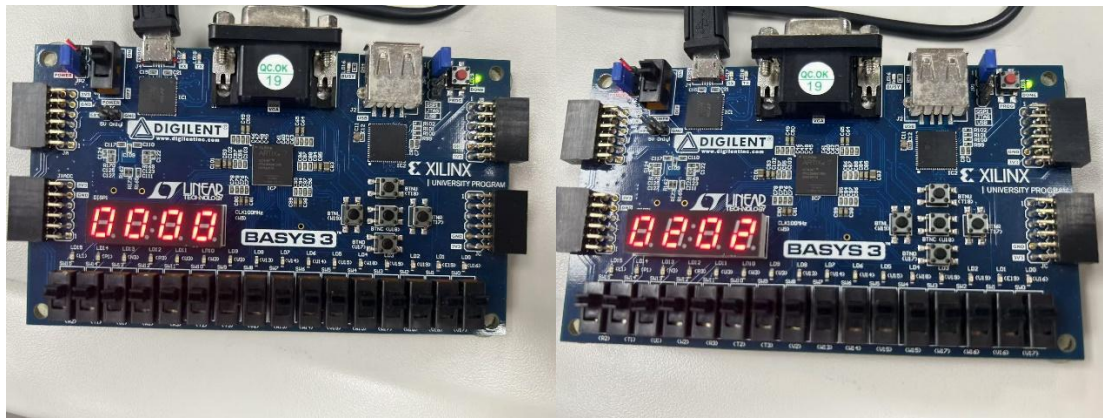
- Rt 的编号和内容





- ALU 输出和总线输出





六 实验心得

这次单周期 CPU 的实验经历了不少困难，但也让我收获了很多宝贵的经验和知识。在实验过程中，我遇到了很多挑战，尤其是在设计控制信号时，由于没有正确初始化一些信号，导致仿真始终无法通过，反复调试了很久才发现问题所在。此外，在设计过程中，我曾错误地添加了一个额外的输入，结果出现了大问题，花费了大量时间去排查。很高兴最后还是在自己的努力下解决了问题。通过这次实验，我不仅加深了对单周期 CPU 设计原理的理解，学习了一个新的语言 verilog，还提高了自己解决问题的能力，学会了如何在面对困难时保持冷静和耐心，逐步排查并解决问题。